

Attention (cont.)

CSE 5526 - Intro to Neural Networks
Lecture 19

Instructor: Donald S. Williamson

 @TheASPIREGrpOSU

 theaspiregroup_osu

 The ASPIRE Group @ OSU

Agenda and Learning Outcomes

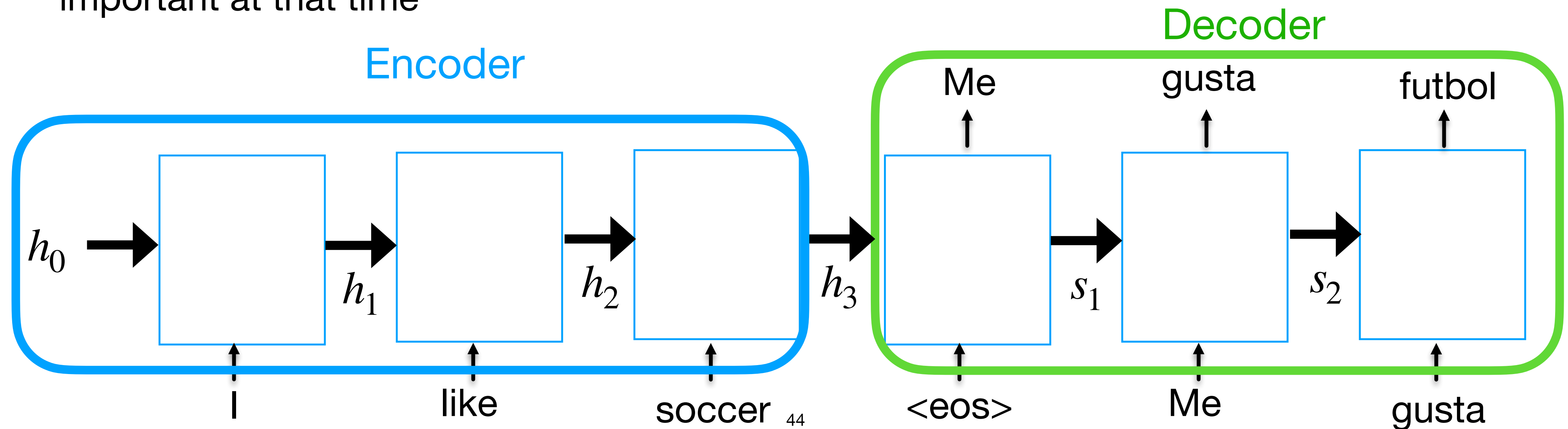
Upcoming Topics

- **Topics:**
 - Recap Attention
 - More examples of Attention
 - Discuss General Attention
 - Introduce Self Attention (time permitting)
- **Announcements**
 - Quiz #8 posted on Thursday (on NMT encoder-decoder, and attention), due by Tuesday, April 1st.
 - Programming assignment #2 has been posted. Due by April 17th for full credit.
 - An optional bonus programming assignment will also be posted next week.

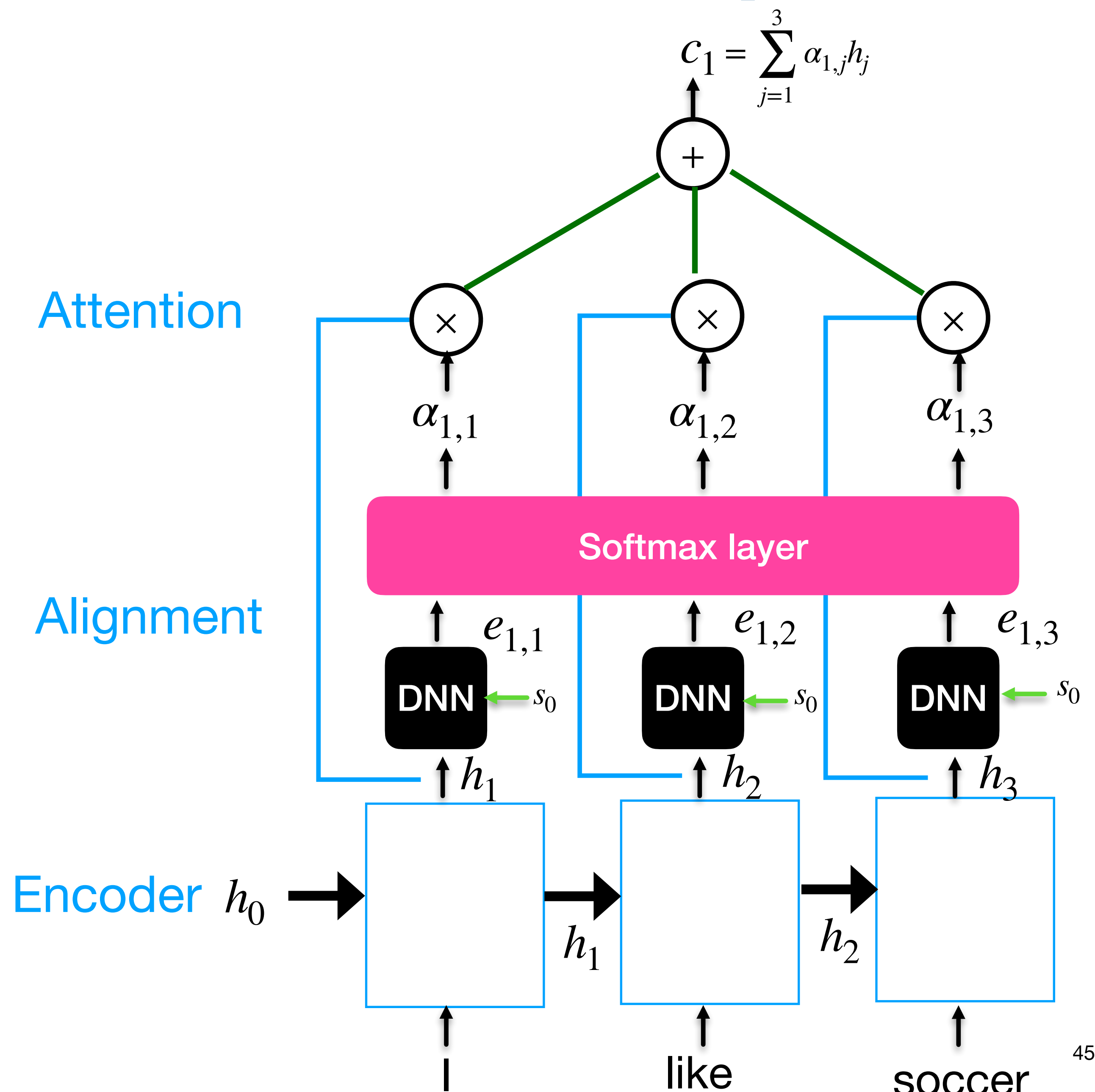
Recap: Encoder-Decoder NMT

Limitations

- Their performance is good, but there are issues with this approach:
 - A single fixed-length vector (e.g., last hidden state of the encoder), must represent everything about the source text
 - Key information must be “passed” to words at the end of the target sequence, where this can be a long path (e.g., think “soccer” to “futbol”)
 - The decoder at each time step cannot focus on aspects of the source sentence that are most important at that time

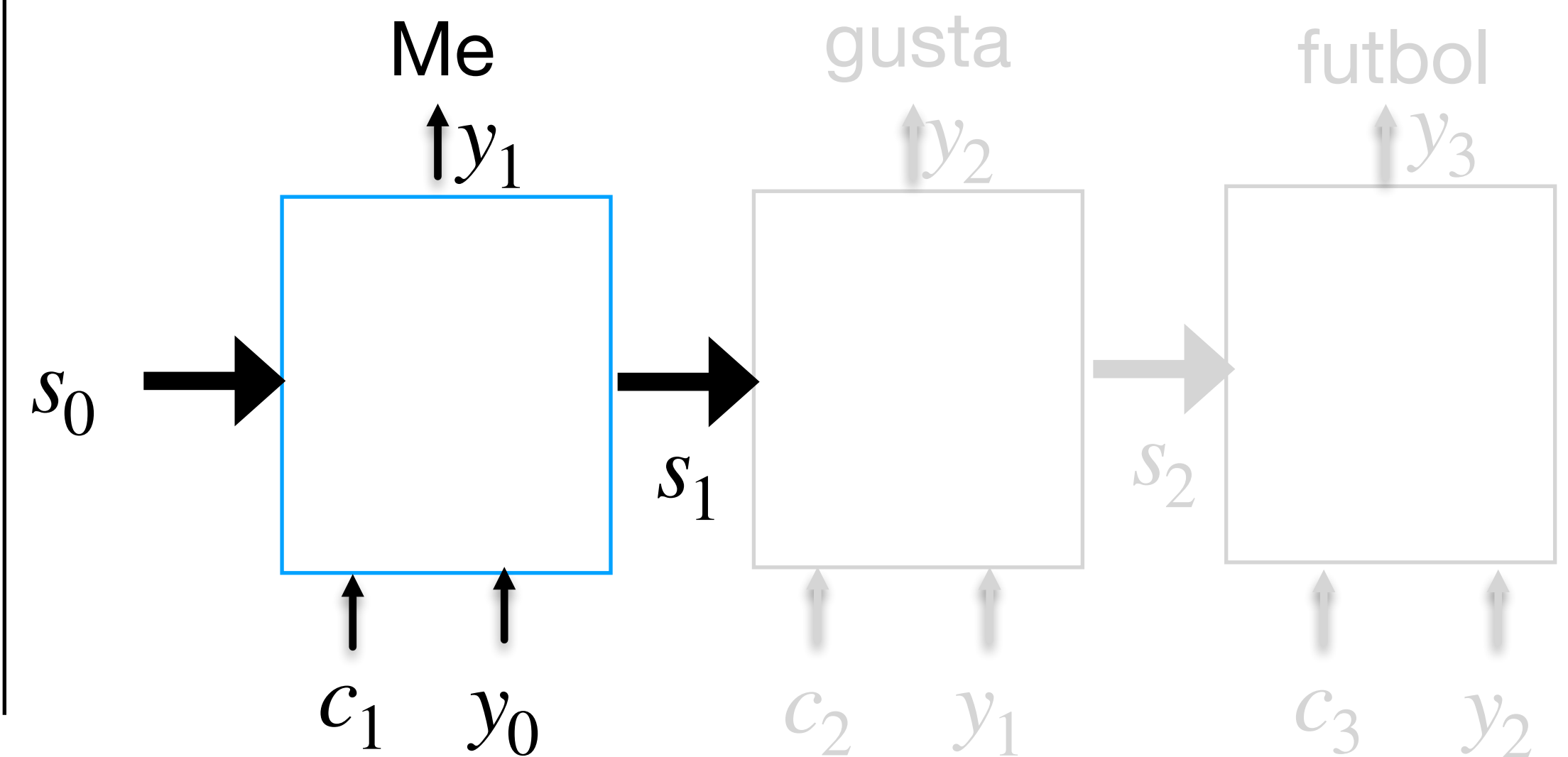


Attention Example

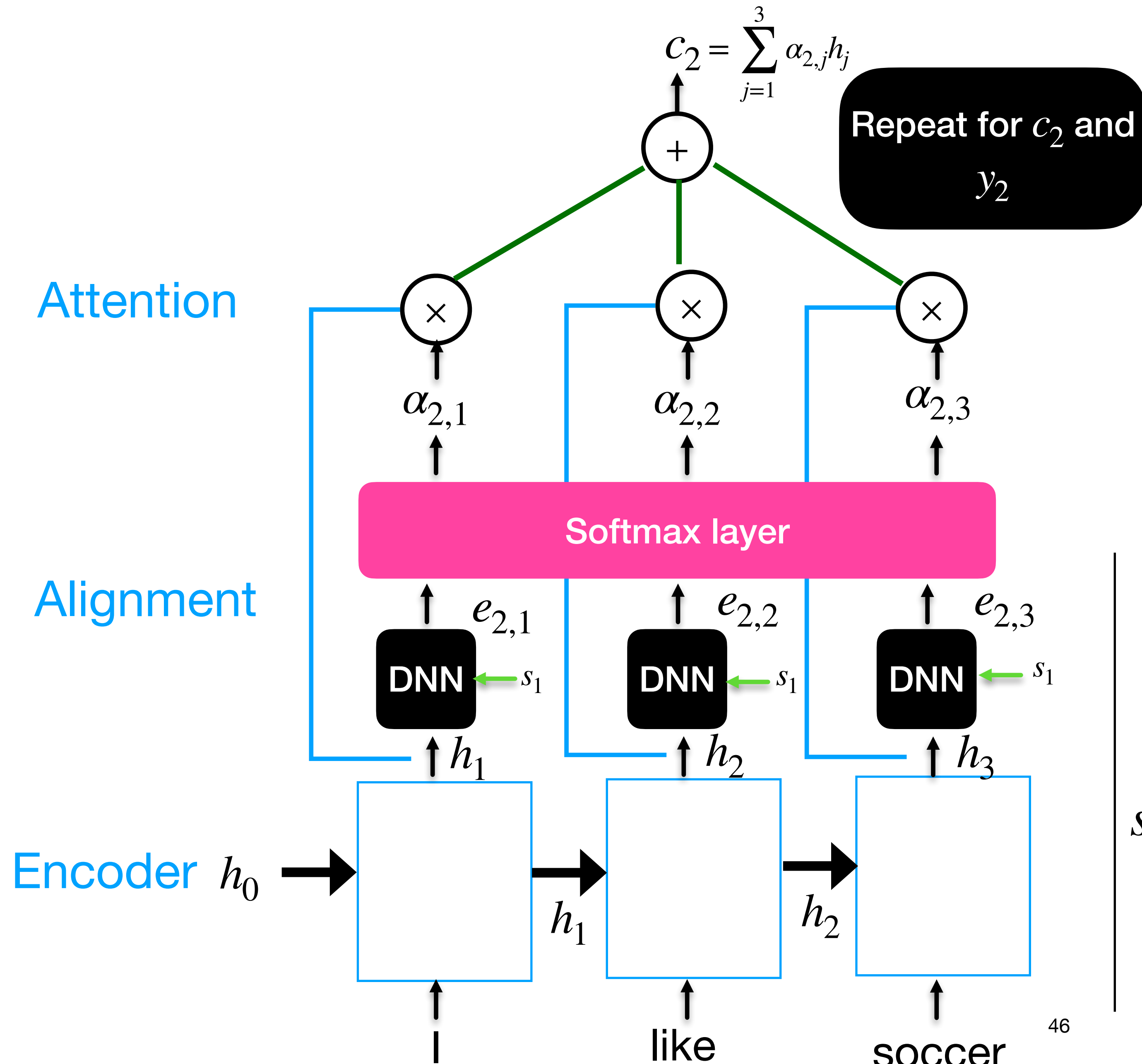


- Want the decoder to output first element of sequence in Spanish (e.g., “Me”)
- Compute c_1 using the encoder, alignment, and attention operations.
 - Compute h_1, h_2, h_3 using the encoder
 - Here, $i = 1$, so $s_{i-1} = s_0 = \mathbf{0}$ (vector of zeros)
 - Compute the alignment scores.
 $e_{1,1} = \text{DNN}(h_1, s_0), \quad e_{1,2} = \text{DNN}(h_2, s_0), \quad e_{1,3} = \text{DNN}(h_3, s_0)$
 - Compute the attention weights $\alpha_{1,1}, \alpha_{1,2}, \alpha_{1,3}$
 - Compute the context vector, c_1
- Provide this as the input to the decoder. Then compute the decoder outputs.

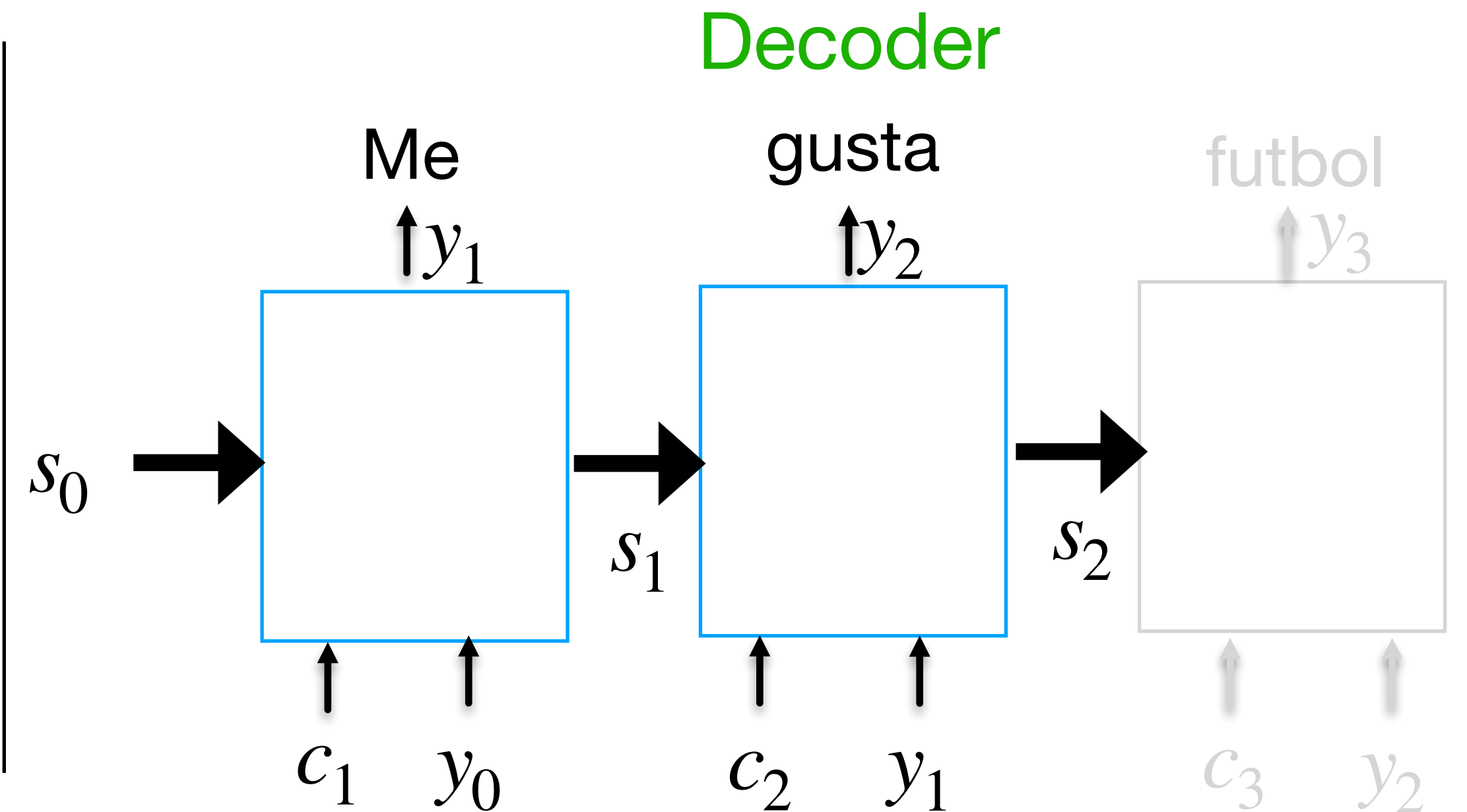
Decoder



Attention Example



- Want the decoder to output 2nd element of sequence in Spanish (e.g., "gusto")
 - Compute c_2 using the encoder, alignment, and attention operations.
 - Compute h_1, h_2, h_3 using the encoder
 - Here, $i = 2$, so $s_{i-1} = s_1$ (1st hidden state of decoder)
 - Compute the alignment scores.
 $e_{2,1} = \text{DNN}(h_1, s_1)$, $e_{2,2} = \text{DNN}(h_2, s_1)$, $e_{2,3} = \text{DNN}(h_3, s_1)$
 - Compute the attention weights $\alpha_{2,1}, \alpha_{2,2}, \alpha_{2,3}$
 - Compute the context vector, c_2
 - Provide this as the input to the decoder. Then compute the decoder outputs.



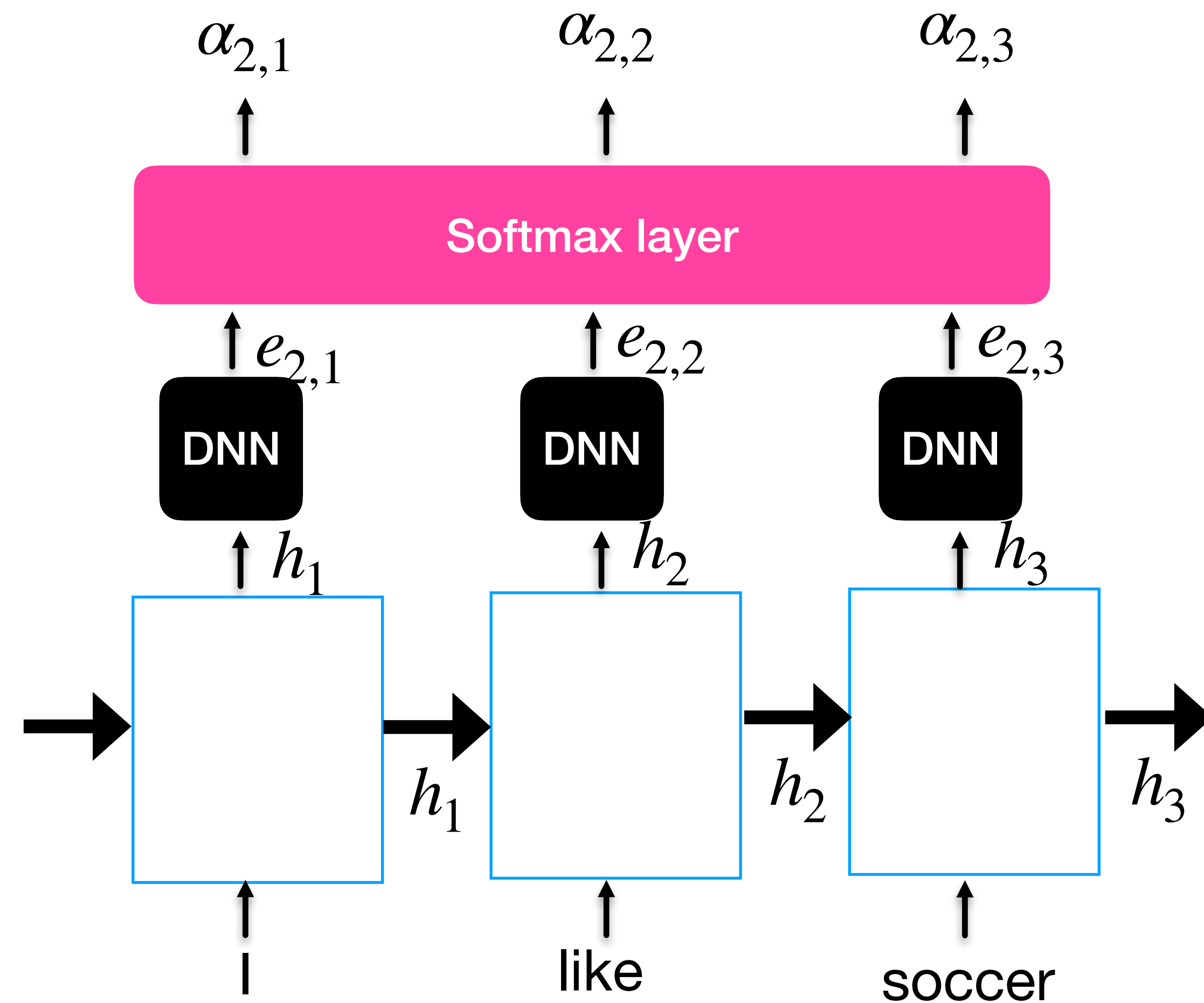
Attention Example

Summary

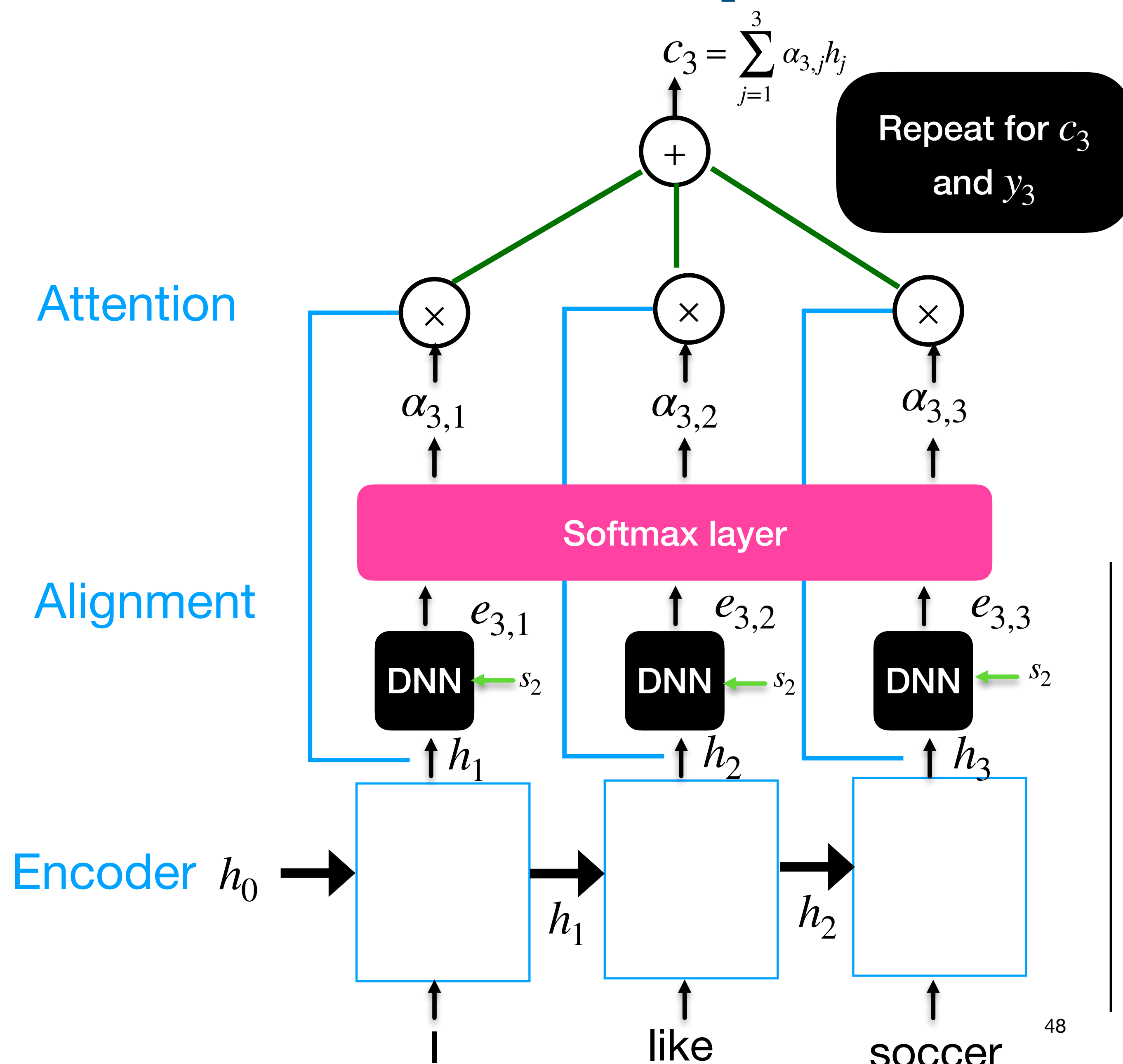
- Likewise, when I'm dealing with the second word of the target sentence. This word **Gusta** is “closest” to the word **like** in the source sentence
- So, the weight, $\alpha_{2,2}$, for the second word should be larger than the weights for the other two words ($\alpha_{2,1}$, $\alpha_{2,3}$)
- Hence, the second element of the decoder's output sequence should attend more to the second hidden state, h_2 , and not the other two states, (h_1 , h_3).

Attention

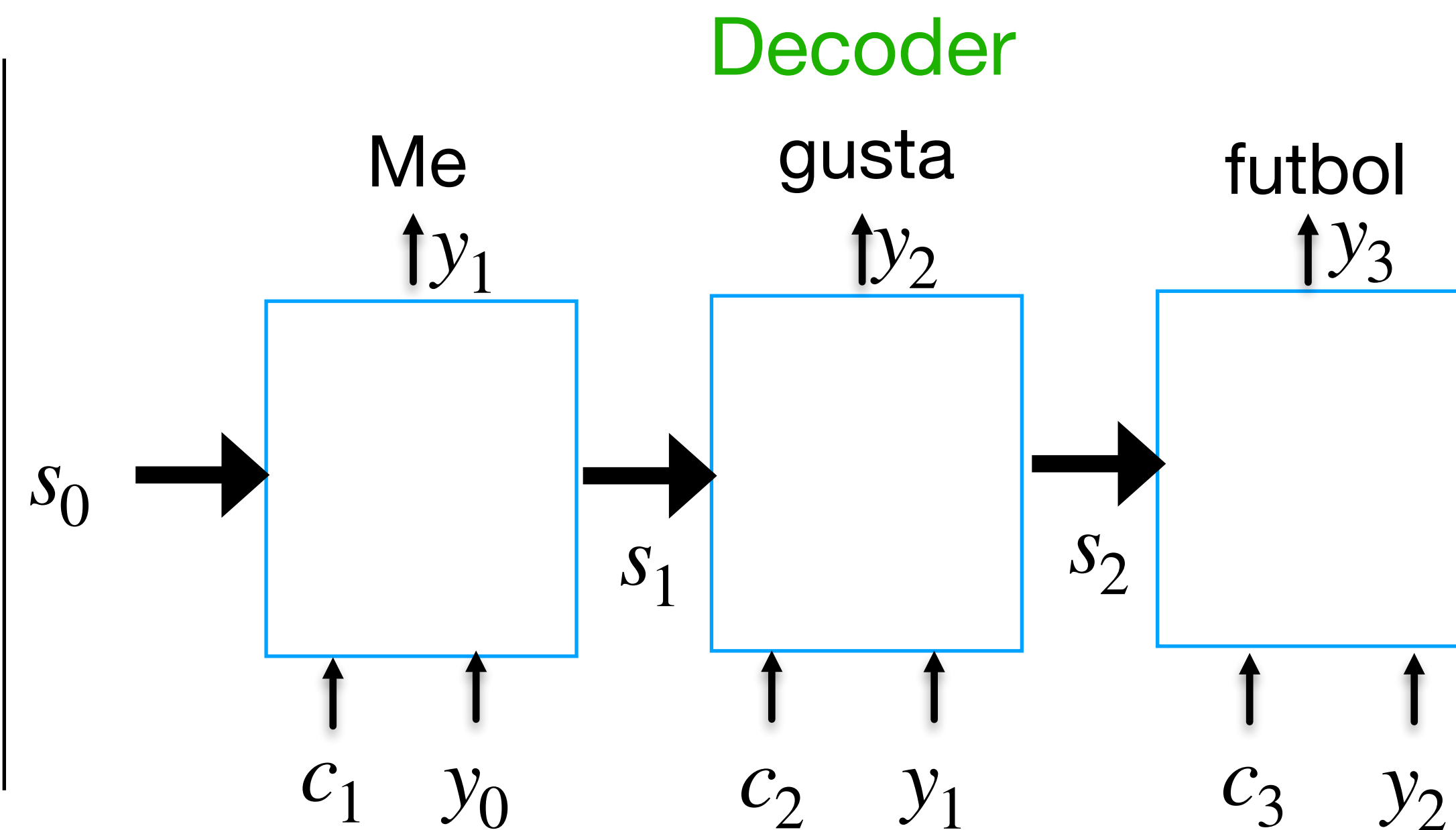
Encoder h_0



Attention Example



- Want the decoder to output 2nd element of sequence in spanish (e.g., “gusto”)
 - Compute c_3 using the encoder, alignment, and attention operations.
 - Compute h_1, h_2, h_3 using the encoder
 - Here, $i = 3$, so $s_{i-1} = s_2$ (second hidden state of decoder)
 - Compute the alignment scores.
 $e_{3,1} = \text{DNN}(h_1, s_2), \quad e_{3,2} = \text{DNN}(h_2, s_2), \quad e_{3,3} = \text{DNN}(h_3, s_2)$
 - Compute the attention weights $\alpha_{3,1}, \alpha_{3,2}, \alpha_{3,3}$
 - Compute the context vector, c_3
 - Provide this as the input to the decoder. Then compute the decoder outputs.



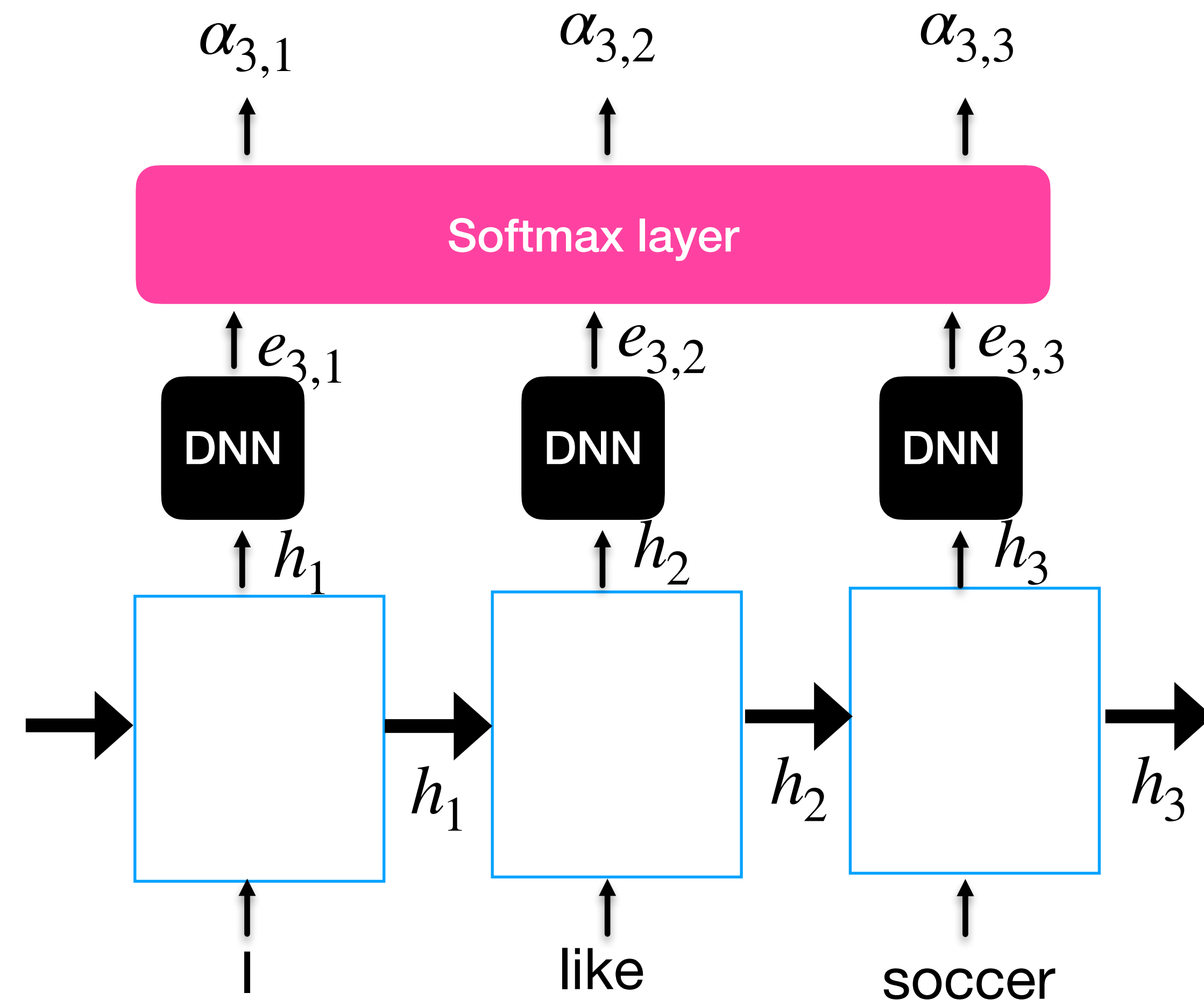
Attention Example

Summary

- Likewise, when I'm dealing with the third word of the target sentence. This word **futbol** is “closest” to the word **soccer** in the source sentence
- So, the weight, $\alpha_{3,3}$, for the second word should be larger than the weights for the other two words ($\alpha_{3,1}, \alpha_{3,2}$)
- Hence, the third element of the decoder's output sequence should attend more to the third hidden state, h_3 , and not the other two states, (h_1, h_2).

Attention

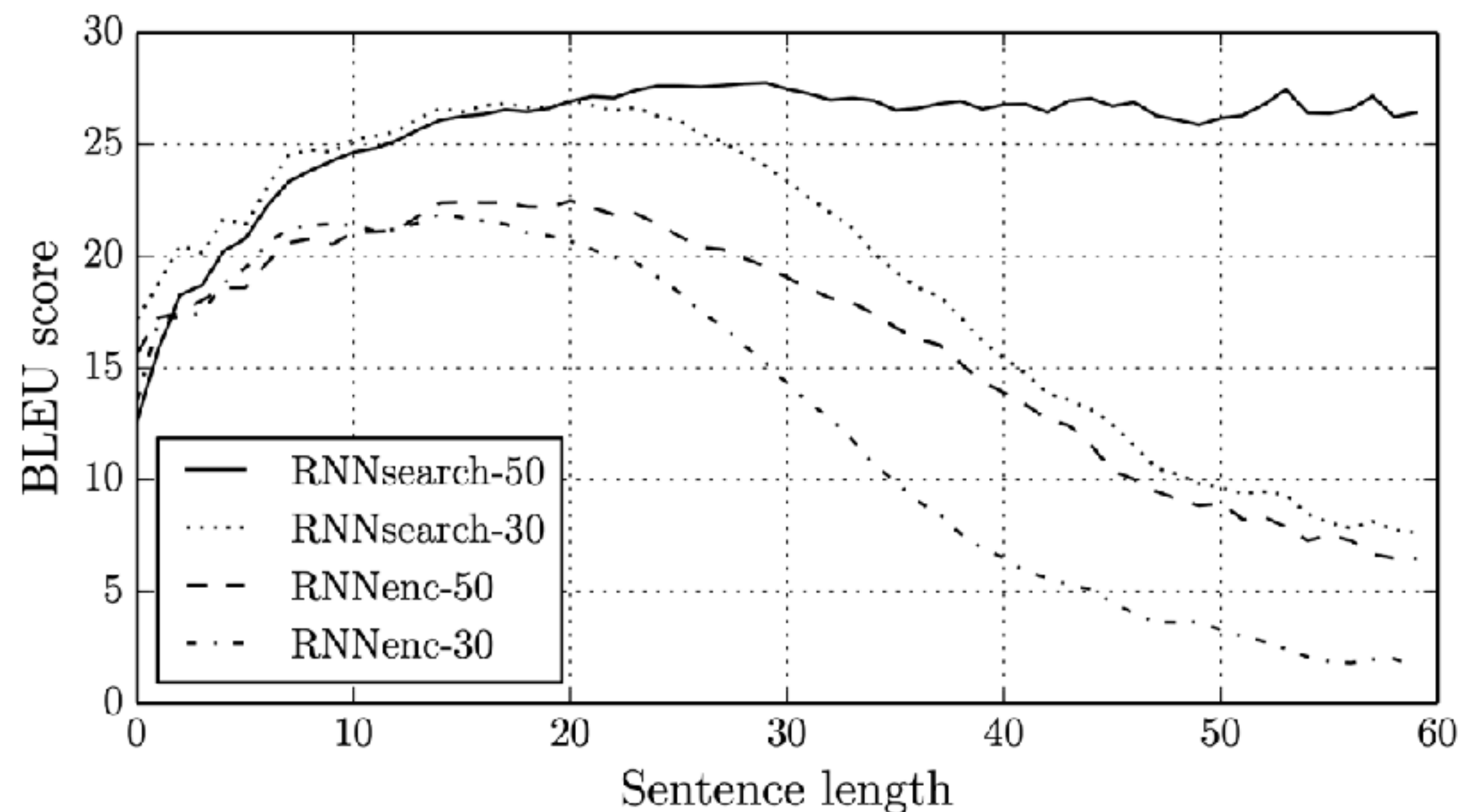
Encoder h_0



The Attention Mechanism

Performance

- Their proposed approach is termed ***RNNsearch-X***, where X is the maximum number of words in the sentence
- The previous RNN encoder-decoder approach (without attention) is called ***RNNencdec***



Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*
Université de Montréal

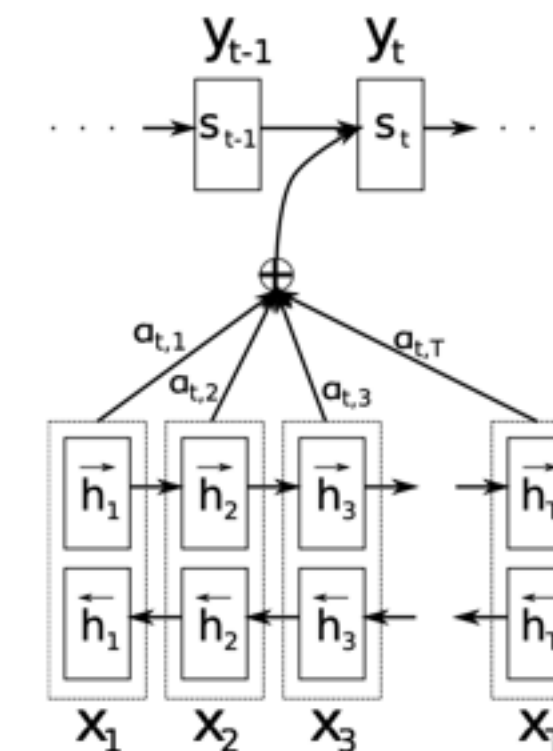


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

Summary

- Attention addresses limitations of Encoder-Decoder RNN based approaches for NMT
 - **RNN:** A single fixed-length vector (e.g., last hidden state of the encoder), must represent everything about the source text
 - **Attention:** Multiple vectors from the encoder are used to each represent a word from the source language
- **RNN:** Key information must be “passed” to words at the end of the target sequence, where this can be a long path (e.g., think “soccer” to “futbol”)
- **Attention:** Information does not have to be passed along long paths
- **RNN:** The decoder at each time step cannot focus on aspects of the source sentence that are most important at that time
- **Attention:** Each time step of the decoder can focus on the specific element of the encoder that is pertinent to it

Questions

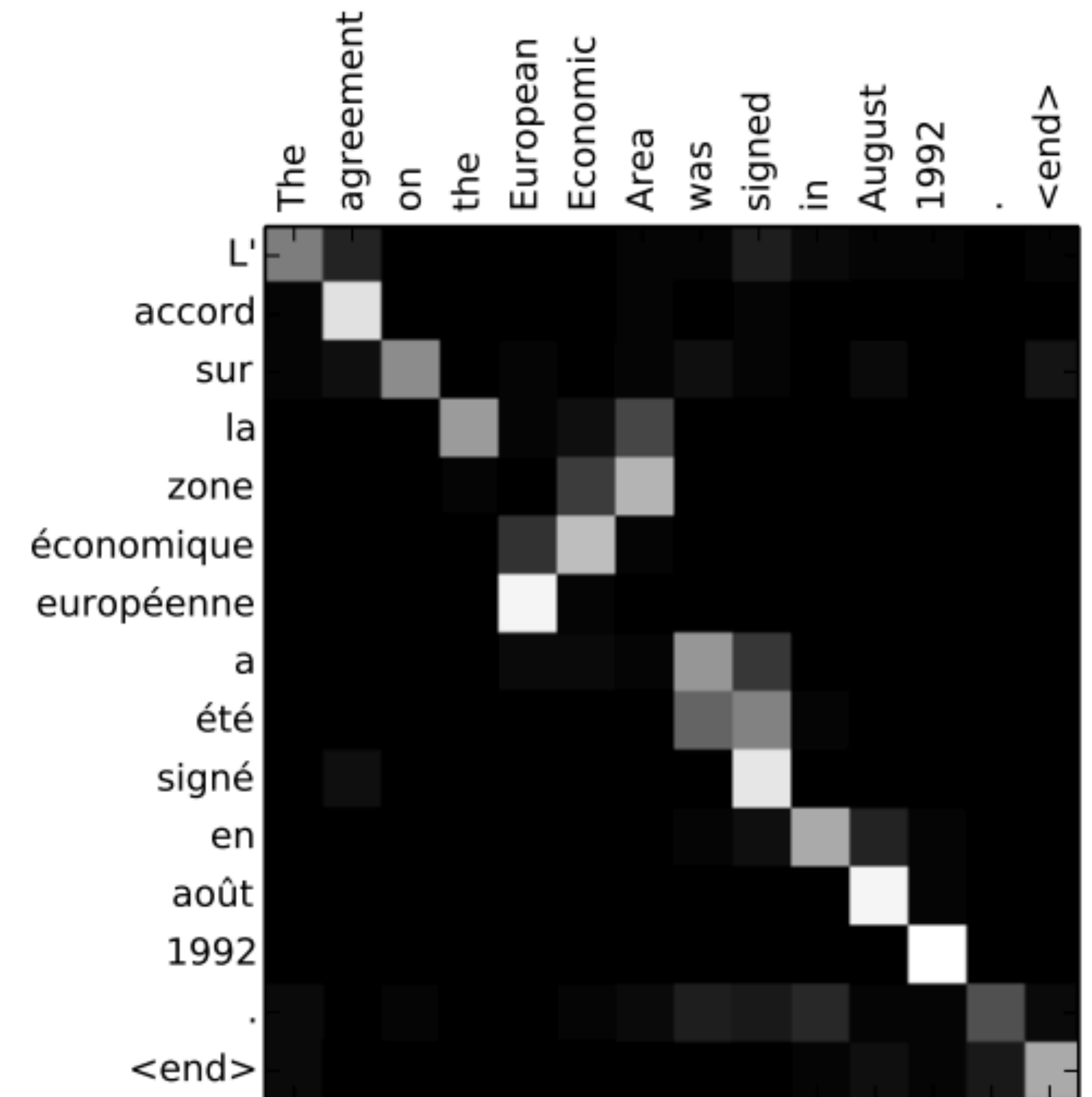
- **Attention From a Conceptual Perspective**
 - **Idea:** ‘attending’ to different parts of the input to determine a corresponding part of the output sequence.
 - This concept will be illustrated using examples from the Computer Vision & NLP domains.

Attention From An Application Perspective

In Natural Language Processing, the task of machine translation is effectively performed by attending to the relevant words in the input sequence at each prediction step.

Figure 3: Four sample alignments found by RNNsearch-50. The x-axis and y-axis of each plot correspond to the words in the source sentence (English) and the generated translation (French), respectively. Each pixel shows the weight α_{ij} of the annotation of the j -th source word for the i -th target word (see Eq. (6)), in grayscale (0: black, 1: white). (a) an arbitrary sentence. (b–d) three

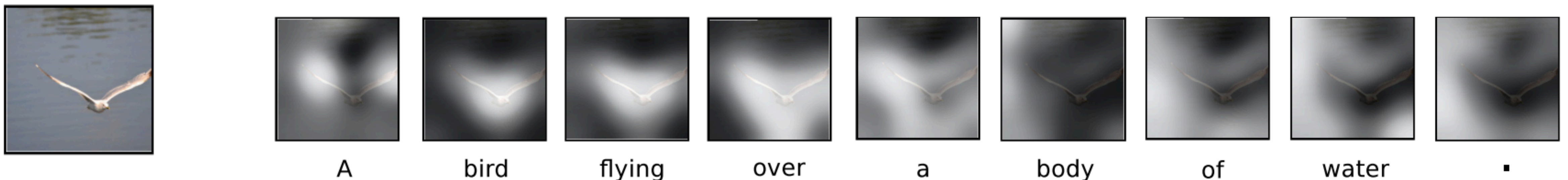
English	French
European	Europeenne
Economic	Economique
Area	Zone



Attention From An Application Perspective

- In Computer Vision, the task of ***image captioning*** is effectively performed by attending to relevant parts of the image when predicting each word.
- In the example below, a model attends to the image of the bird when predicting the label 'bird'. A similar emphasis can be seen on the image of water body when predicting the label 'water'.

Figure 2. Attention over time. As the model generates each word, its attention changes to reflect the relevant parts of the image.



Attention From An Application Perspective

Figure 3. Examples of attending to the correct object (*white* indicates the attended regions, *underlines* indicated the corresponding word)



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



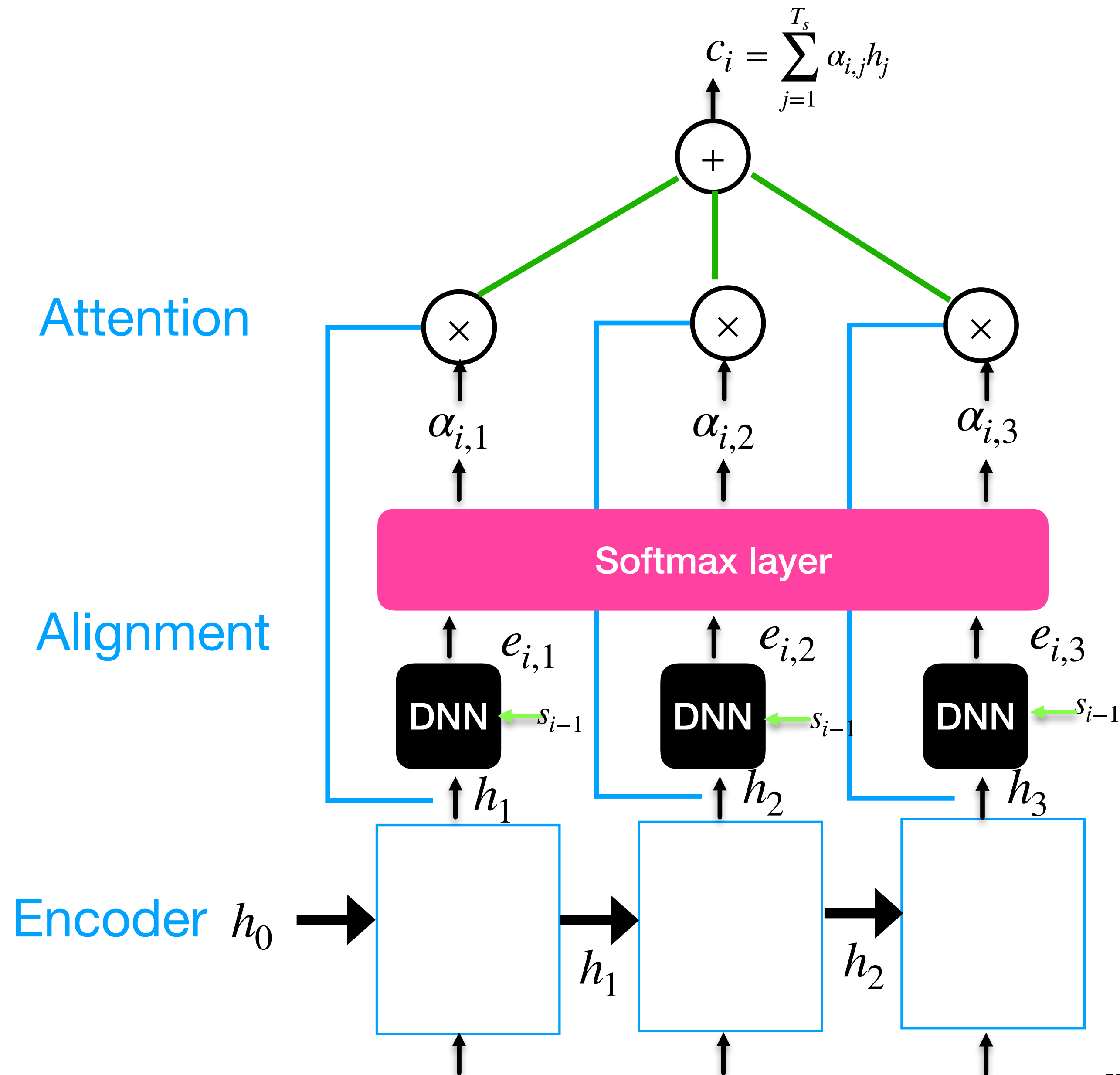
A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

Questions

The Alignment Model: Concatenative Attention

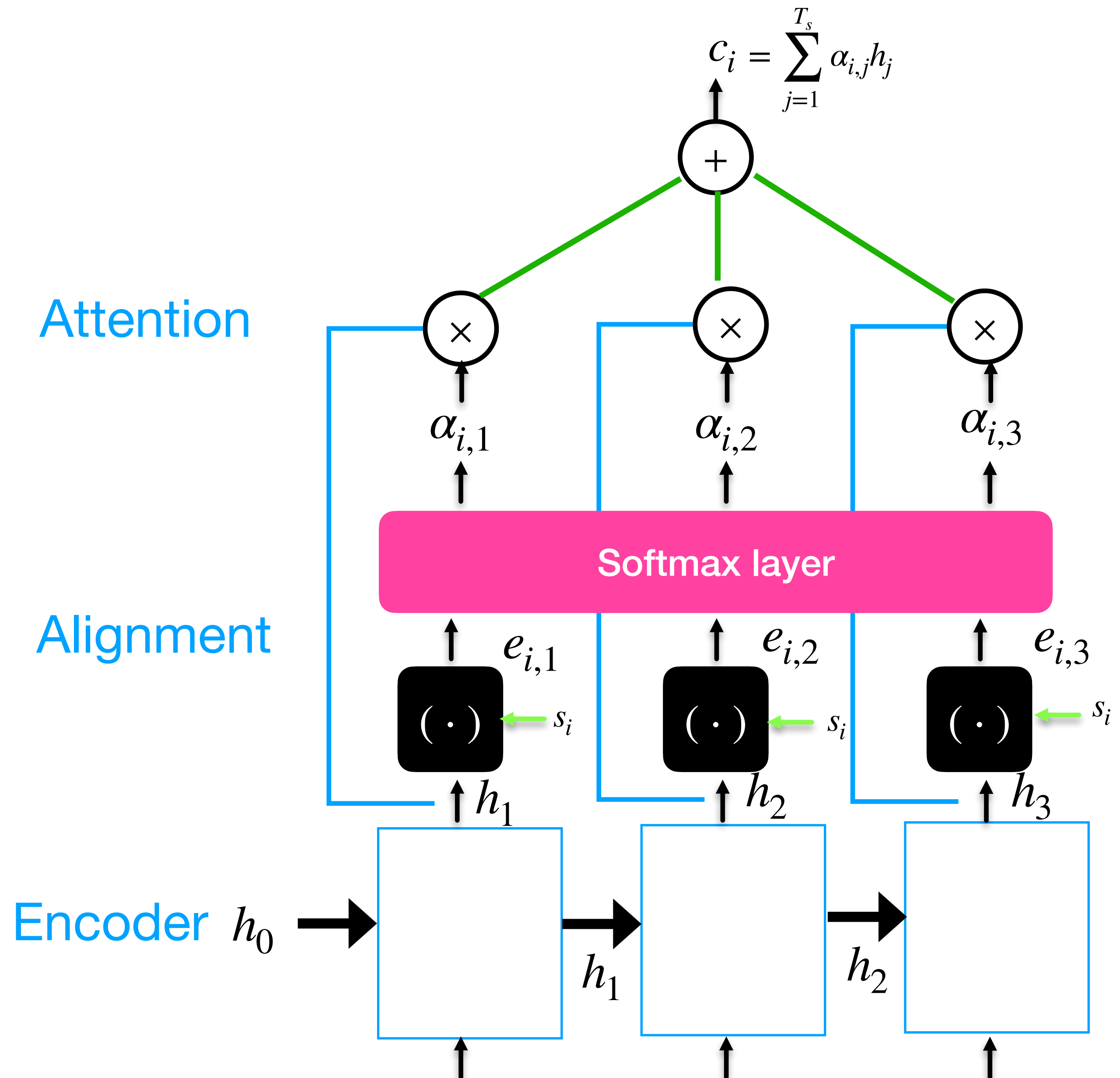


- The alignment model uses a DNN/MLP and the encoder's outputs to produce alignment scores.
- The encoder's output ($\mathbf{h}_j$) is concatenated with the decoder's previous hidden state ($\mathbf{s}_{i-1}$) and provided to the DNN/MLP
- This is sometimes called **concatenative attention**

For example,

$$e_{i,j} = \phi(\mathbf{W}^T[\mathbf{h}_j; \mathbf{s}_{i-1}])$$

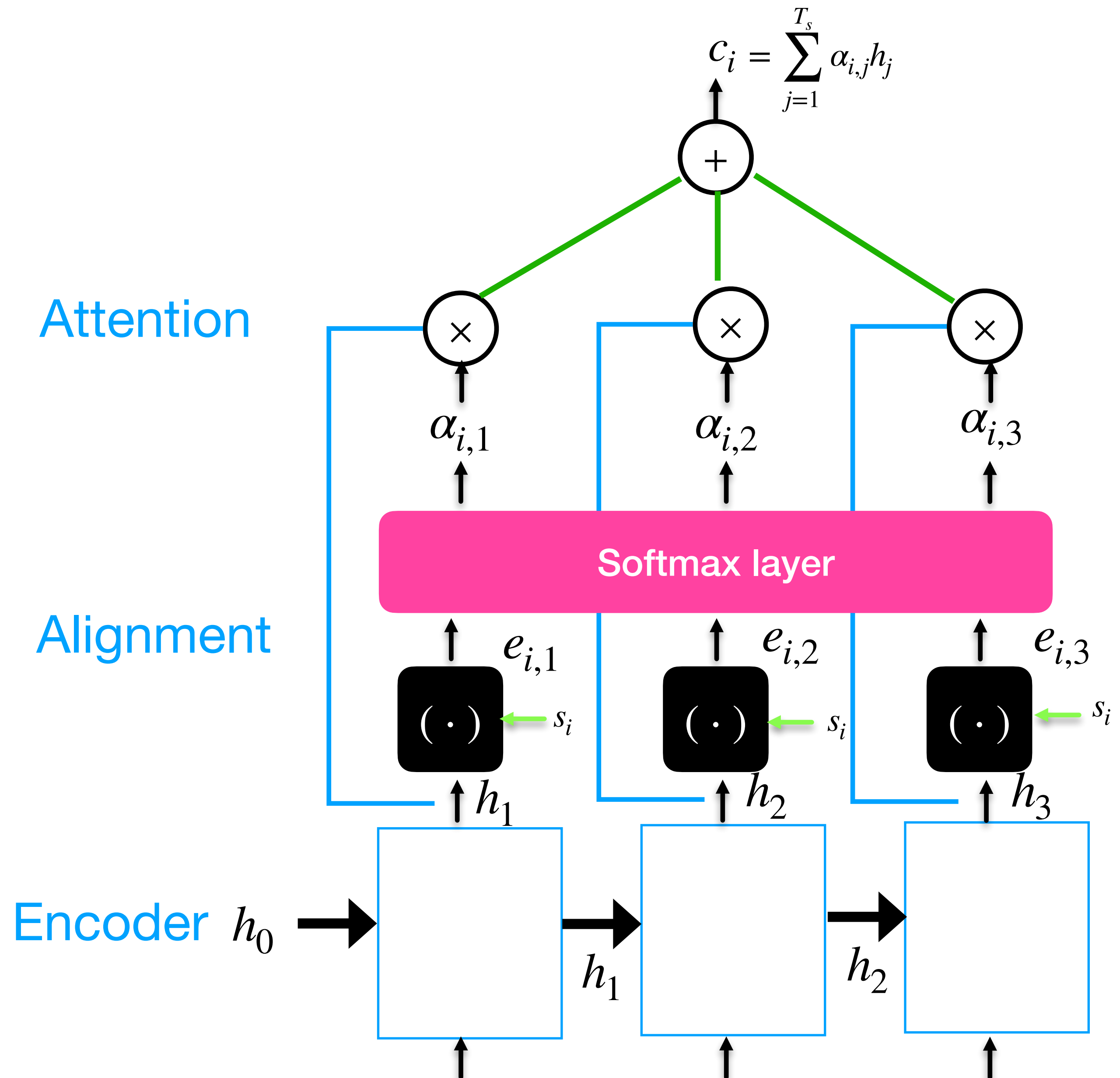
An Alternate Alignment Model: Luong Attention



- The goal of the alignment model is to quantify the similarity between the encoder's output and the decoder's previous hidden state
- The dot product, $(\cdot)$, between two vectors is a fairly good similarity measure
- This is called ***Luong or dot or multiplicative attention***
- ***Luong et al*** also proposed to use the decoder's current hidden state

$$e_{i,j} = \mathbf{h}_j^T \mathbf{s}_i$$

Another Alignment Model: General Attention



- A third alignment model is based on giving the encoder's output to a fully connected linear layer (without a bias) before computing the dot product

- This is called **general attention**

$$e_{i,j} = \mathbf{h}_j^T \mathbf{W} \mathbf{s}_i$$

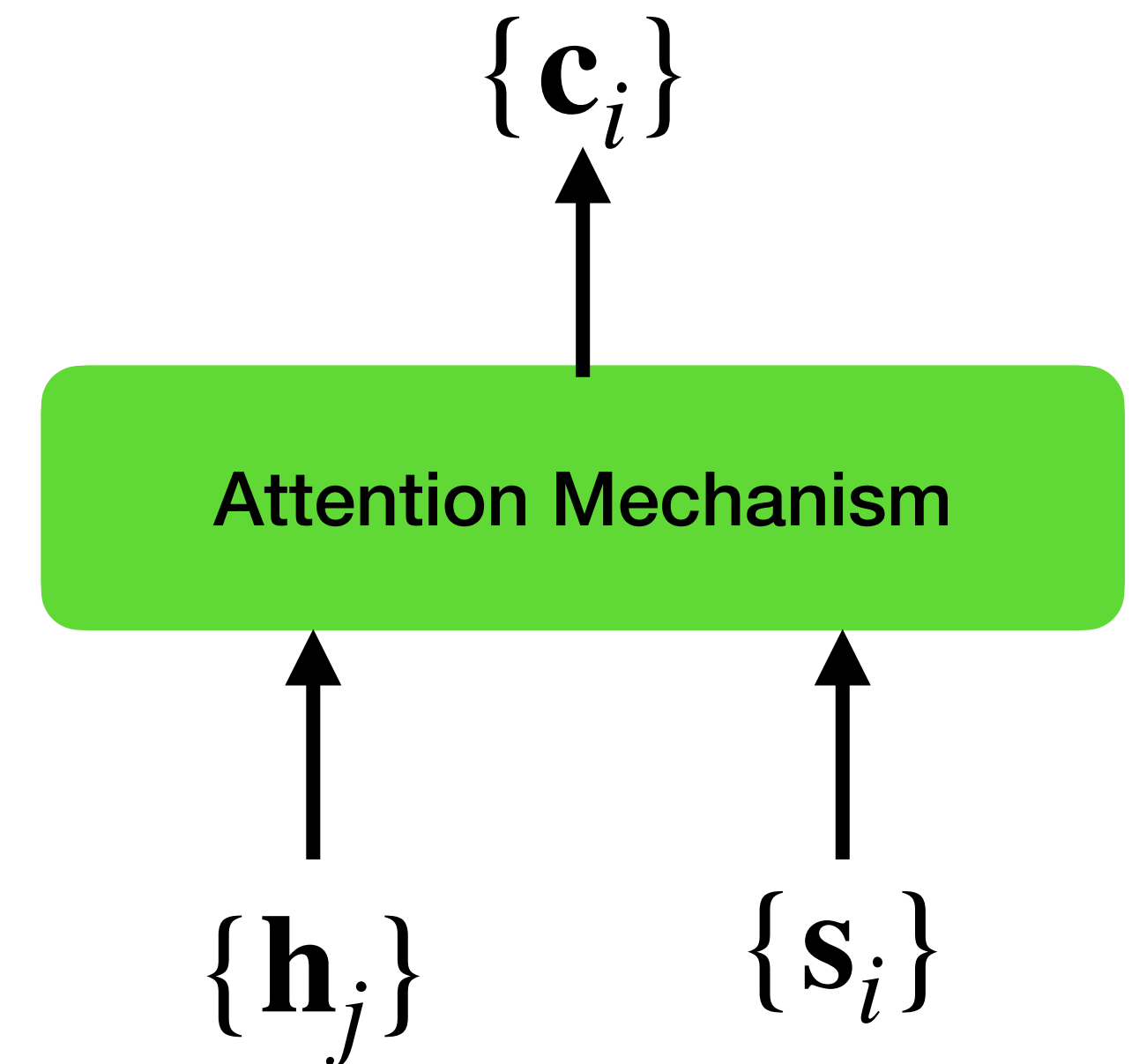
- The researchers found that the dot-product variants (Luong and general) performed best

Questions

Main Components of Attention

For NMT

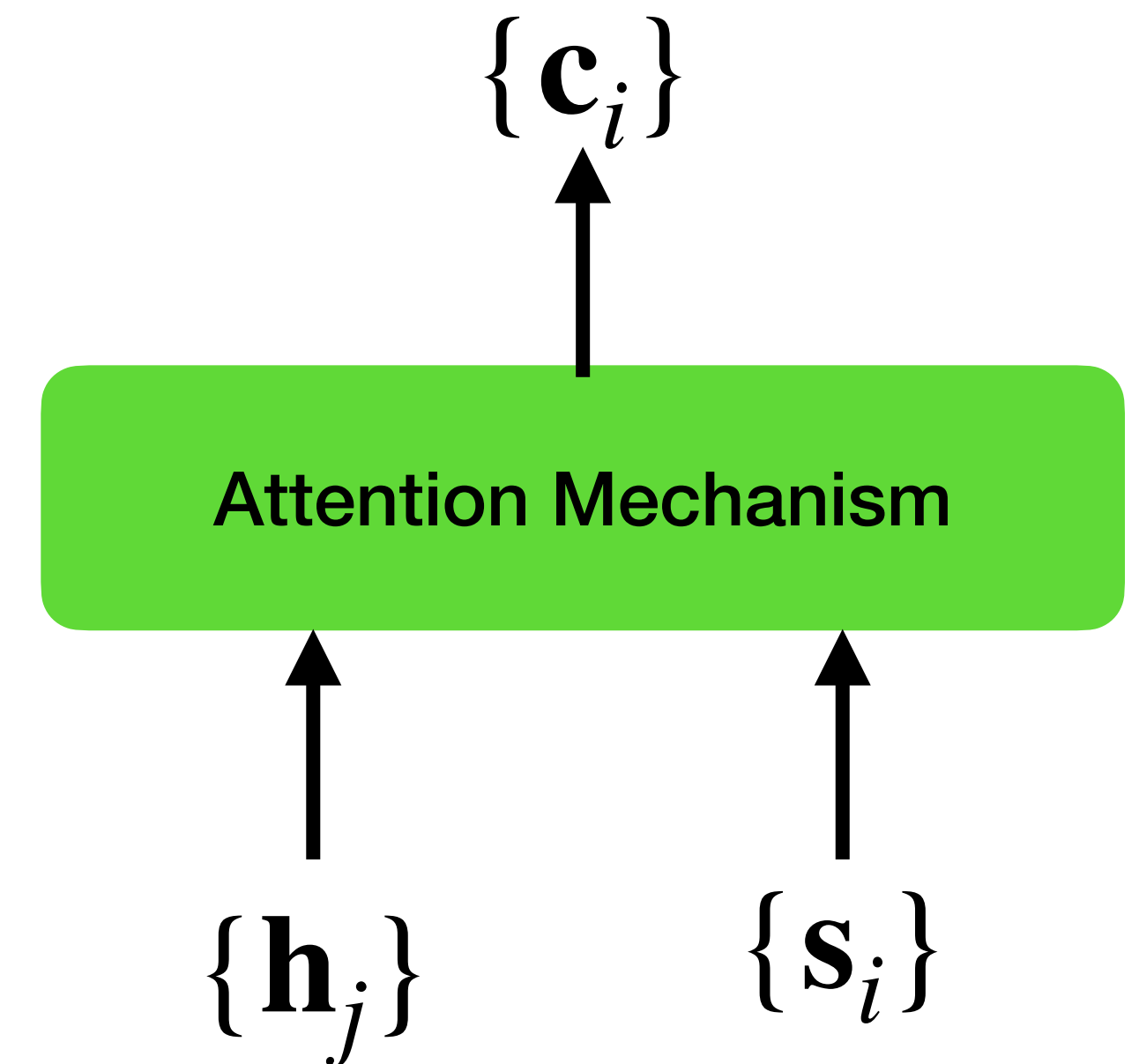
- The attention mechanism has three main representations
 - The decoder's hidden states: $\{\mathbf{s}_i\}$
 - The encoder's hidden states: $\{\mathbf{h}_j\}$
 - The context vectors: $\{\mathbf{c}_i\}$
- Next, we will relate these to three important terms: **key**, **value**, and **query**



Main Components of Attention

Back to the NMT example

- The attention mechanism has three main representations
 - The decoder's hidden states: $\{\mathbf{s}_i\}$
 - This is the *query*
 - The encoder's hidden states: $\{\mathbf{h}_j\}$
 - These are the *keys* and *values*
 - The context vectors: $\{\mathbf{c}_i\}$
 - Computed by linearly combining each weighted value
- Recall, the context vector is based on the alignment/attention between the *keys* and *queries*



More Terminology

From a different example

- You want to find a movie on Netflix. We want to automate this process.
- Each movie has a fix-sized list of attributes (e.g., genre, leading actors, movie length, director,...).
 - The **keys** encode the attributes of each movie, where each movie has a **key vector**
 - The **keys** are copied into **value** vectors, one for each movie (more on this later)
 - The attributes of the movie you are looking for are stored in a **query** vector
- We can solve this with attention!
 - Compare the **query** to all the **keys** to produce attention weights
 - Use the attention weights to scale each **value** and return this to the user

Attention - Different Notation

1. $\mathbf{x}_j$ refers to the encoder's hidden state (previously was $\mathbf{h}_j$)

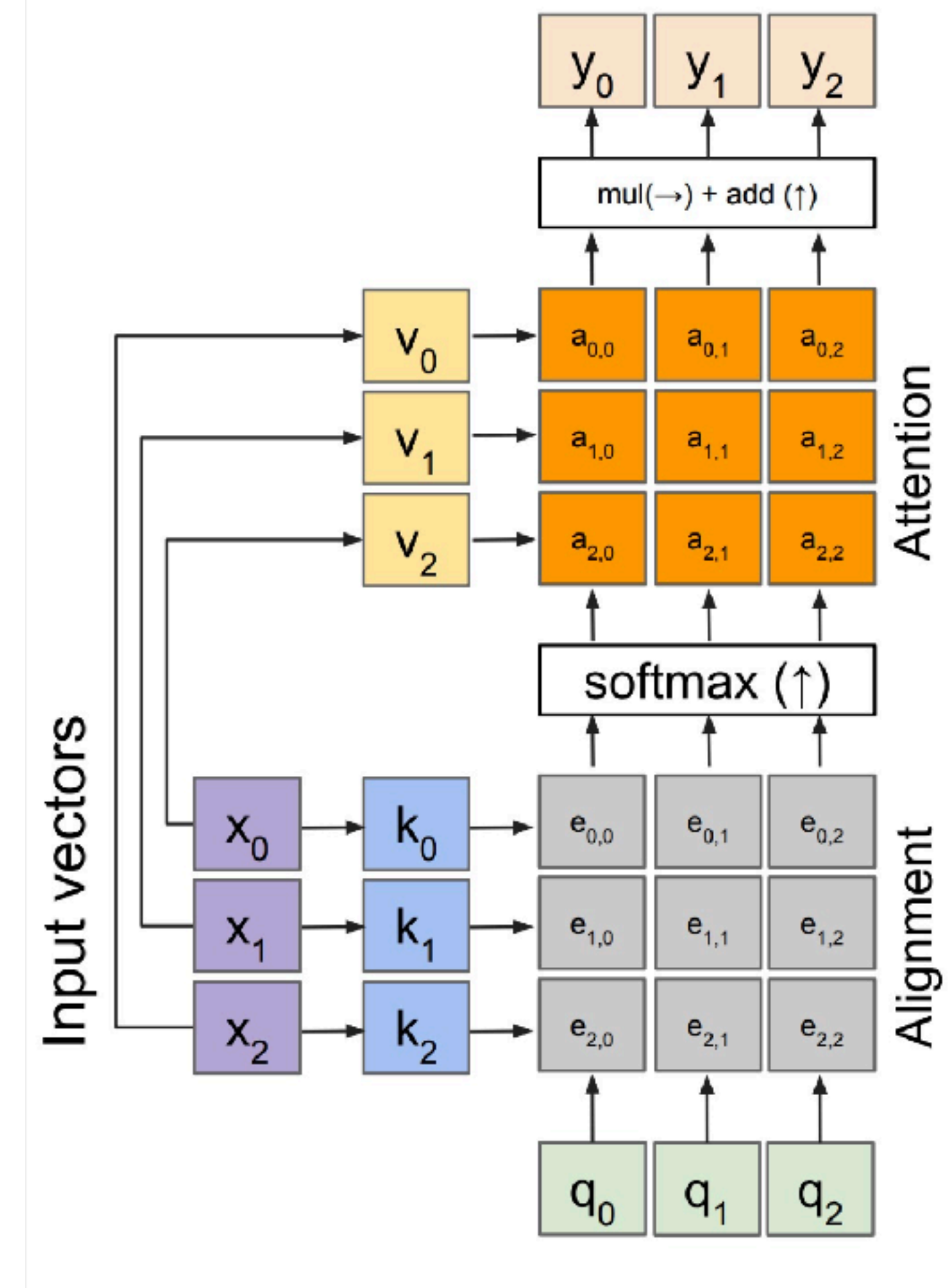
We copy this to two vectors (or matrices):

- The key: $\mathbf{k}_j = \mathbf{x}_j$ (e.g., encoder's hidden states)
- The value: $\mathbf{v}_j = \mathbf{x}_j$

2. The Queries, $\mathbf{q}_i$

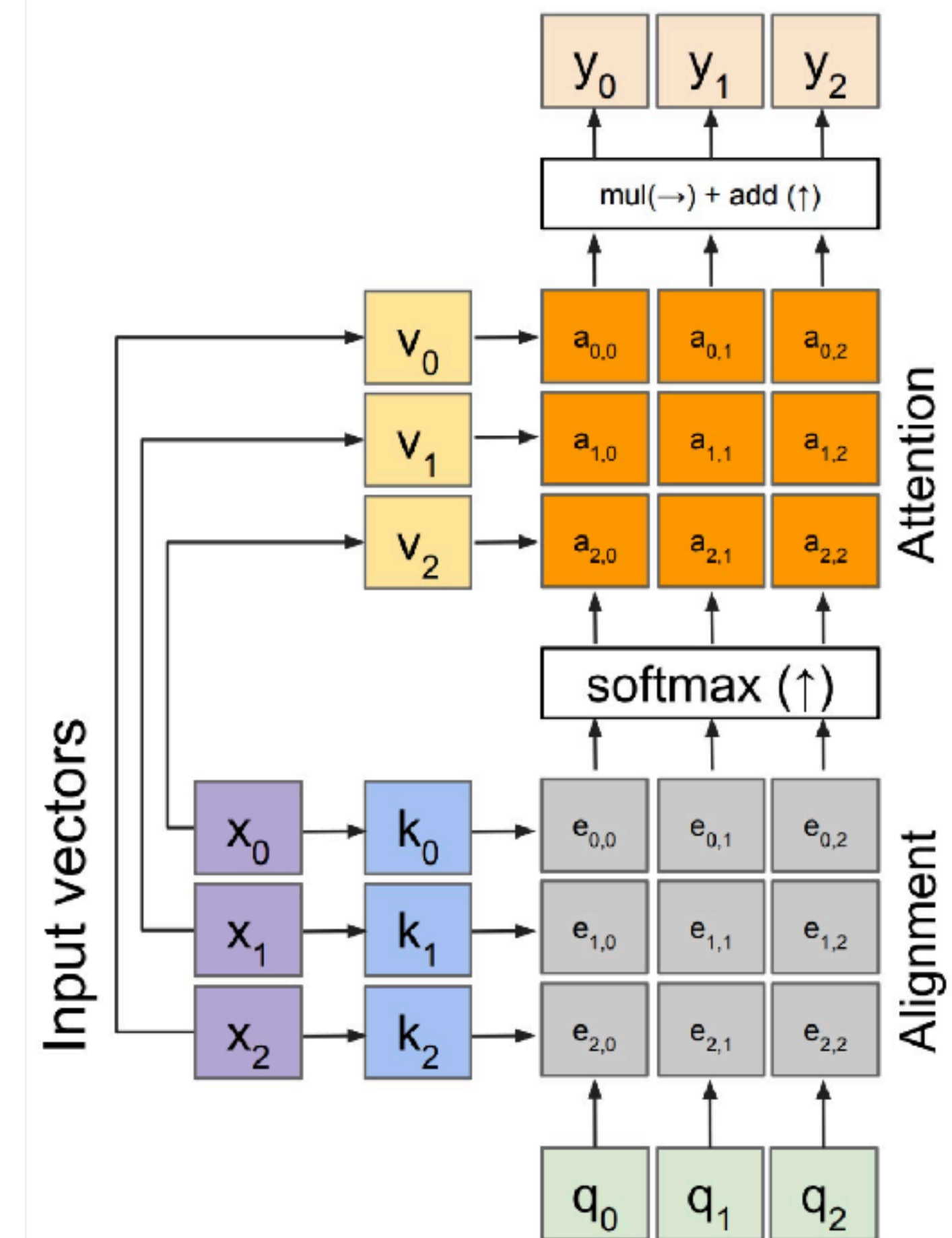
In an RNN decoder, a query can be thought of as hidden state, which interacts with input to generate context representation.

3. The alignment score is computed using the dot product
4. The attention weights are computed within each column (e.g., sum to one within a column)
5. It outputs a context representation, $\mathbf{y}_i$, at each index of the decoder



General Attention

- This approach underscores the basic approach known as **General Attention**
- The query, keys, and values are computed in the general sense, likely involving vectors that come from different data (e.g., encoder and decoder states)
- We can make this more general, by separately linearly transforming the encoder's hidden state to produce the **keys** and **values**
 - Keys: $\mathbf{k}_j = \mathbf{x}_j^T \mathbf{w}_k$
 - Values: $\mathbf{v}_j = \mathbf{x}_j^T \mathbf{w}_v$
- This gives the model more freedom (or expressivity) to learn what matters most



General Attention

3. **Alignment:** The Query-Key interaction is given as a simple dot product.

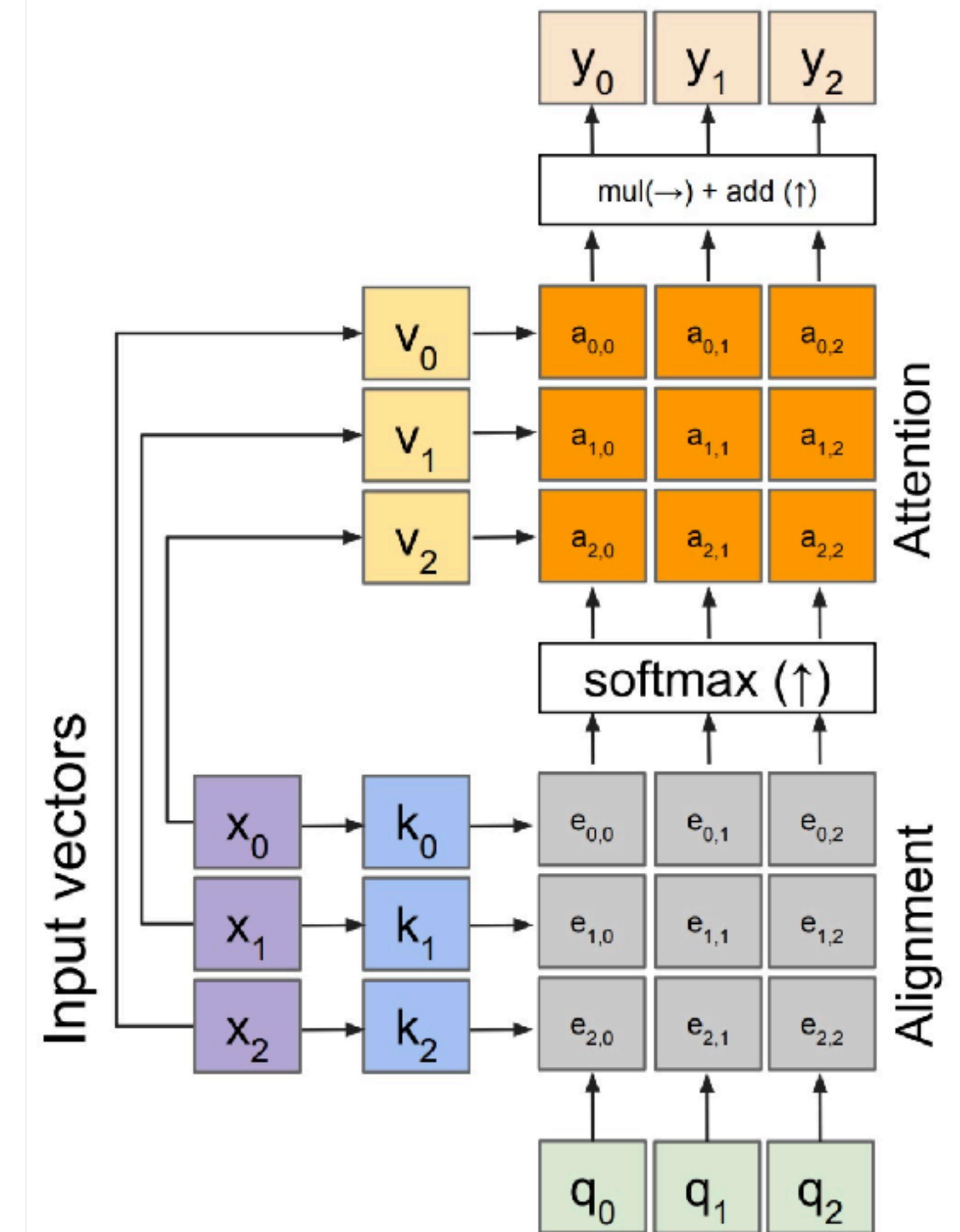
$$e_{ij} = \mathbf{q}_j \cdot \mathbf{k}_i$$

4. **Attention:** Softmax function is applied to the alignment vectors to get attention weights that sum up to 1 (within a column).

$$\mathbf{a} = \text{softmax}(\mathbf{e})$$

5. **Output:** Attention weights are applied to the value vectors to get the final output.

$$\mathbf{y}_j = \sum_i a_{ij} * \mathbf{v}_i$$



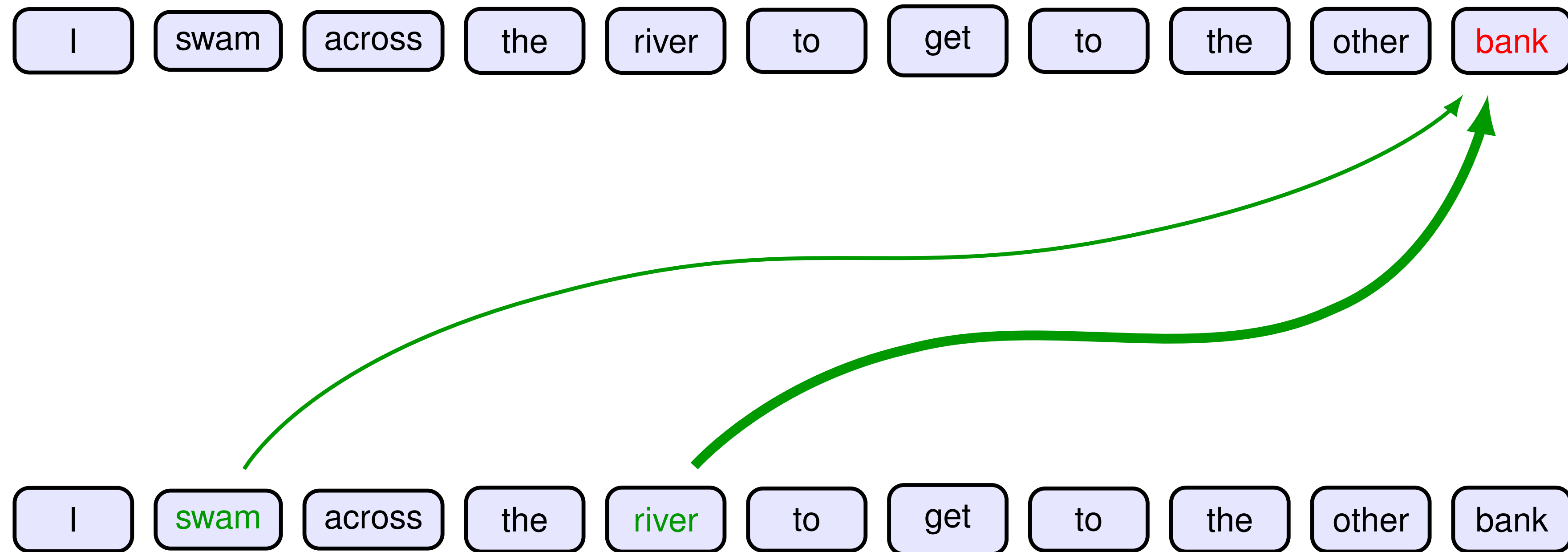
Questions

Self Attention

A different example

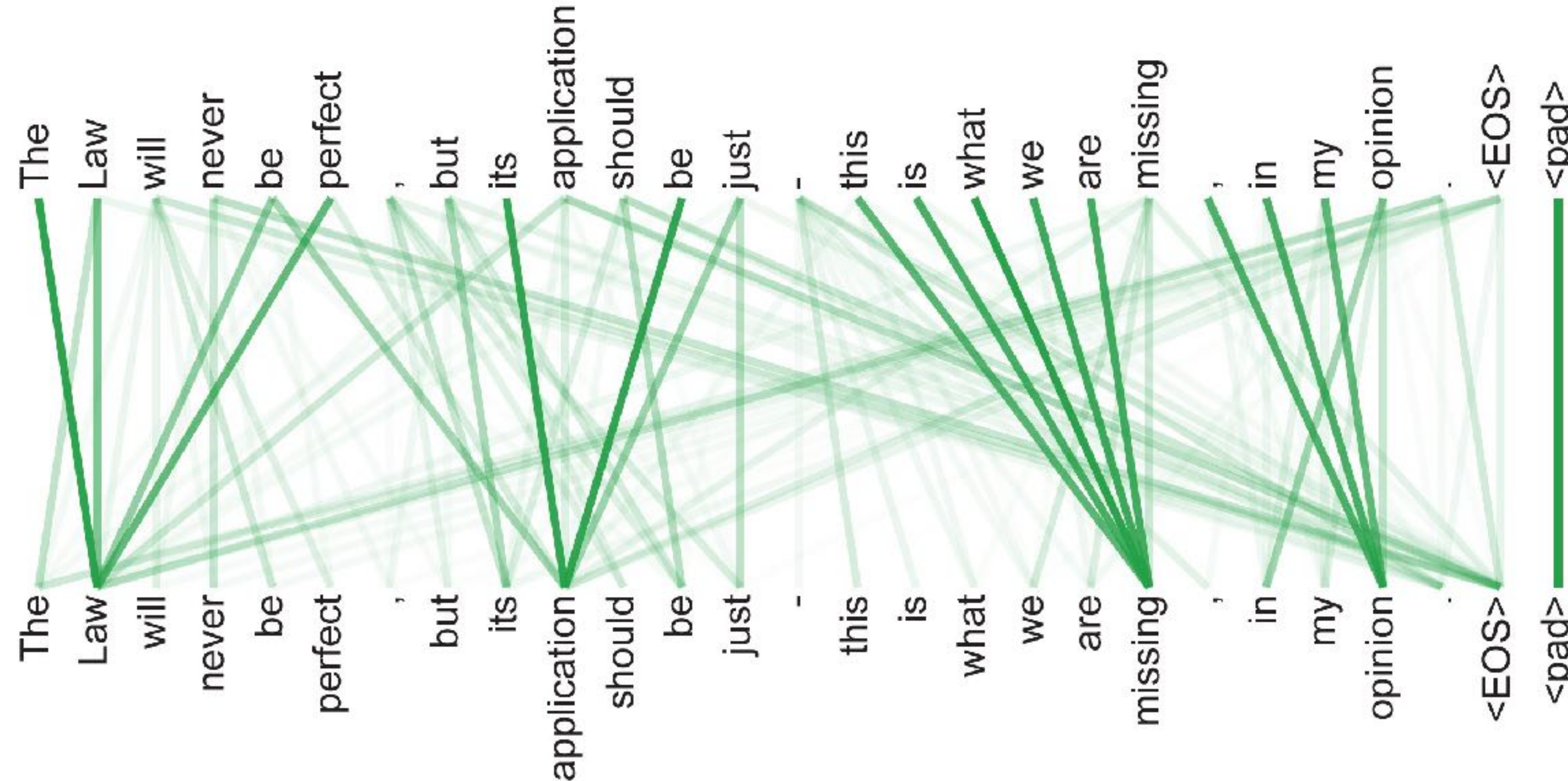
- Consider these two sentences:
 - I swam across the river to get to the other **bank**.
 - I walked across the road to get cash from the **bank**.
- Here, **bank** has a different meaning in each sentence.
- Hence, the interpretation depends on the other words in the sentence.
- This is what ***self attention*** does. It relates different components of the sequence to each other, in the form of attention

An Illustration of Self-Attention



- The lines indicate which words from the sentence influence the definition for **bank**
- The thickness of each line indicates the strength of the word's influence

A Complete Illustration of Self-Attention

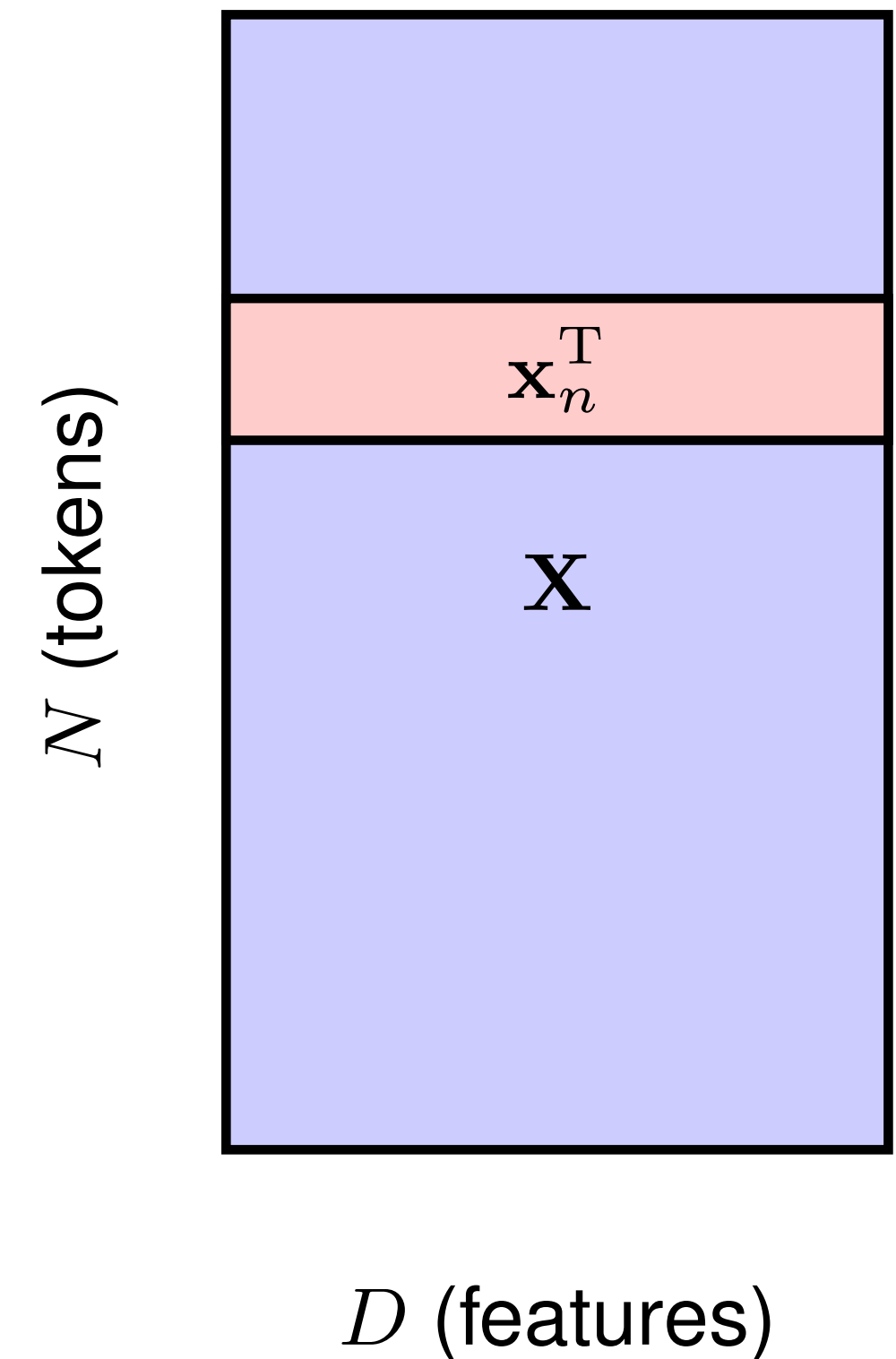


- The lines indicate which words from a sentence influence other words in the sentence
- The thickness of each line indicates the strength of the word's influence

Self Attention

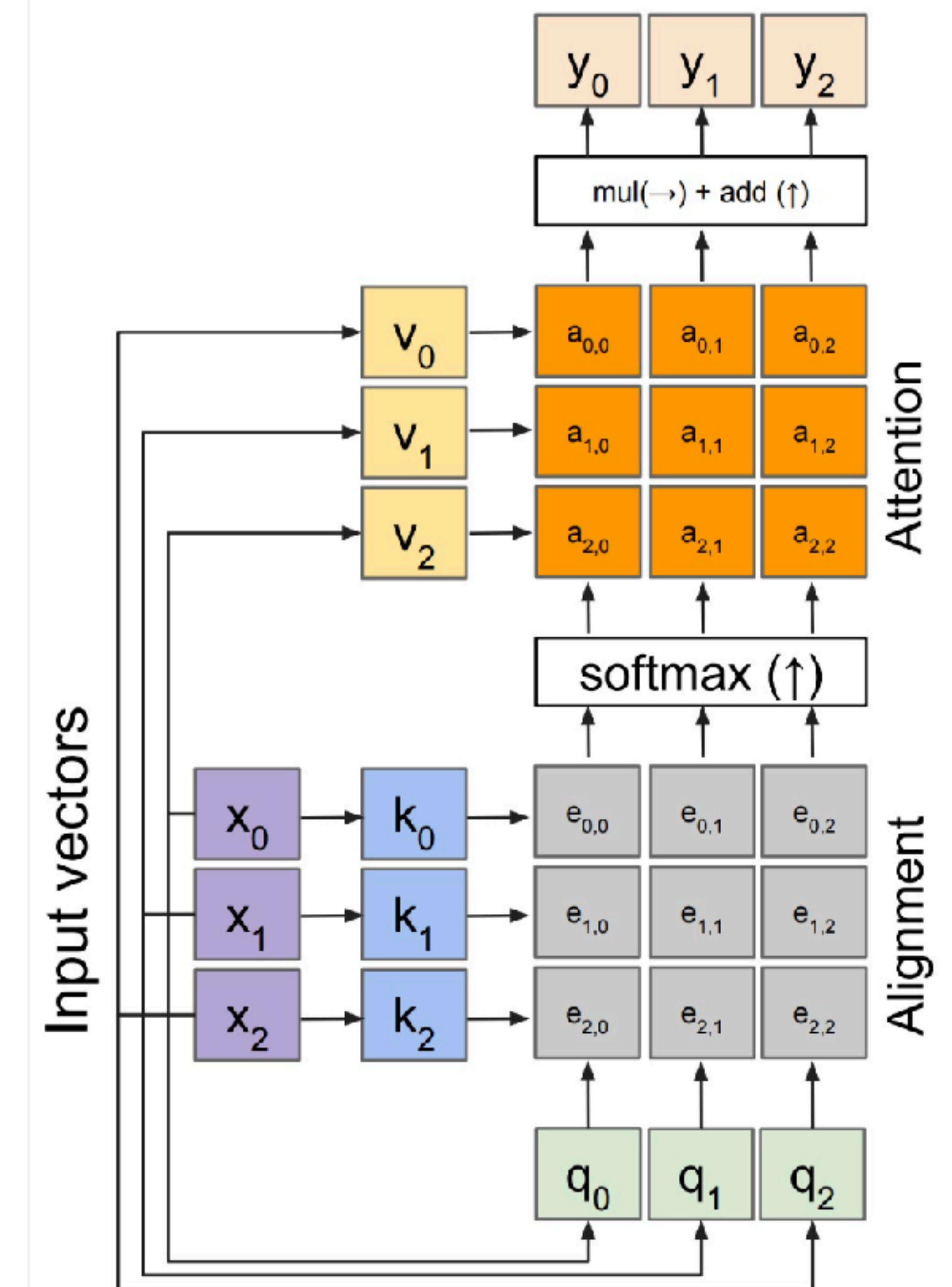
Goal and Notation

- **Goal:** Given a set of **tokens** (e.g., which can be computed from words), we need to compute **attention weights** $\{\alpha_{i,j}\}$ between each pair of **tokens**
 - We will assume that we have N total tokens for a input data sample (e.g., a token for each word in the sentence).
 - We will represent the n^{th} token by $\mathbf{x}_n$, where each token is D -dimensional and each dimension corresponds to a feature
 - All tokens for a sample can be stored in a $N \times D$ matrix, $\mathbf{X}$, where each row corresponds to a token vector



Self Attention

- General attention takes input and query vectors separately as input, whereas the **self-attention mechanism** calculates the query, key, and value vectors from the same input. Hence, the name *self-attention*.
- The query vectors are determined using a learnable weight matrix W_q of dimension $D \times D_q$
 - Query: $Q = XW_q$
 - Each query vector attends to all key vectors to result in an attention map, which when applied to value vectors results in output **context**.



Self Attention

1. Input of dimension $N \times D$.

Ex., given a RGB image of shape $28 \times 28 \times 3$. It is serialized so $N = 28 * 28$ pixels (number of input vectors) and $D = 3$ channels (dimensionality of each vector).

2. Query, Key, and Value vectors/matrices

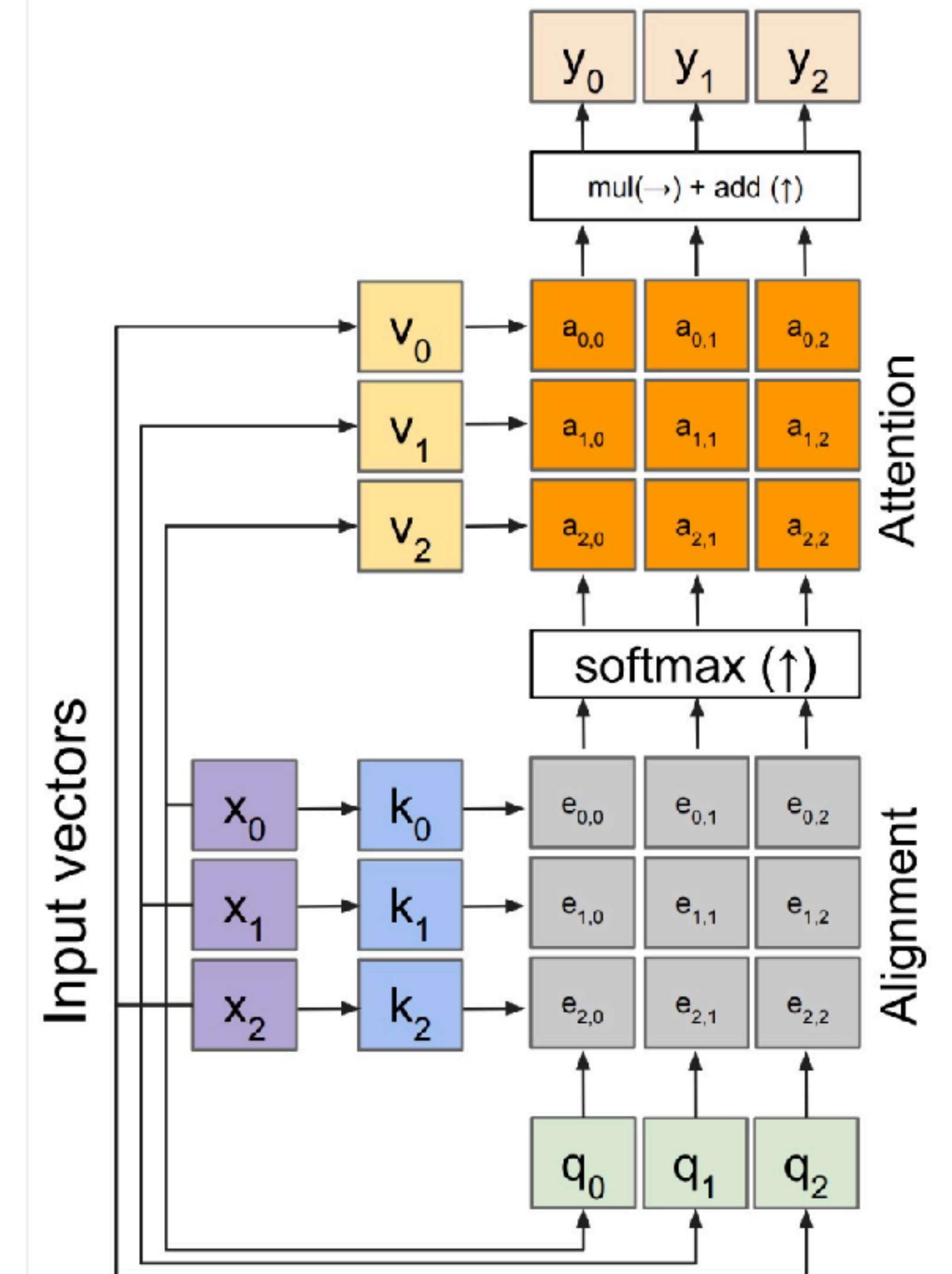
1. Query: $\mathbf{Q} = \mathbf{XW}_q$ of dimension $N \times D_q$

2. Key: $\mathbf{K} = \mathbf{XW}_k$ of dimension $N \times D_k$

3. Value: $\mathbf{V} = \mathbf{XW}_v$ of dimension $N \times D_v$

- $\mathbf{W}_q$ is a learnable query weight matrix of dimension $D \times D_q$
- $\mathbf{W}_k$ is a learnable key weight matrix of dimension $D \times D_k$
- $\mathbf{W}_v$ is a learnable value weight matrix of dimension $D \times D_v$
- In practice, we use the same dimensionality for all the vectors as well.

$$D_q = D_k = D_v$$



Self Attention

3. **Alignment:** The Query-Key interaction is given as a simple dot product.

$$e_{ij} = \mathbf{q}_j \cdot \mathbf{k}_i$$

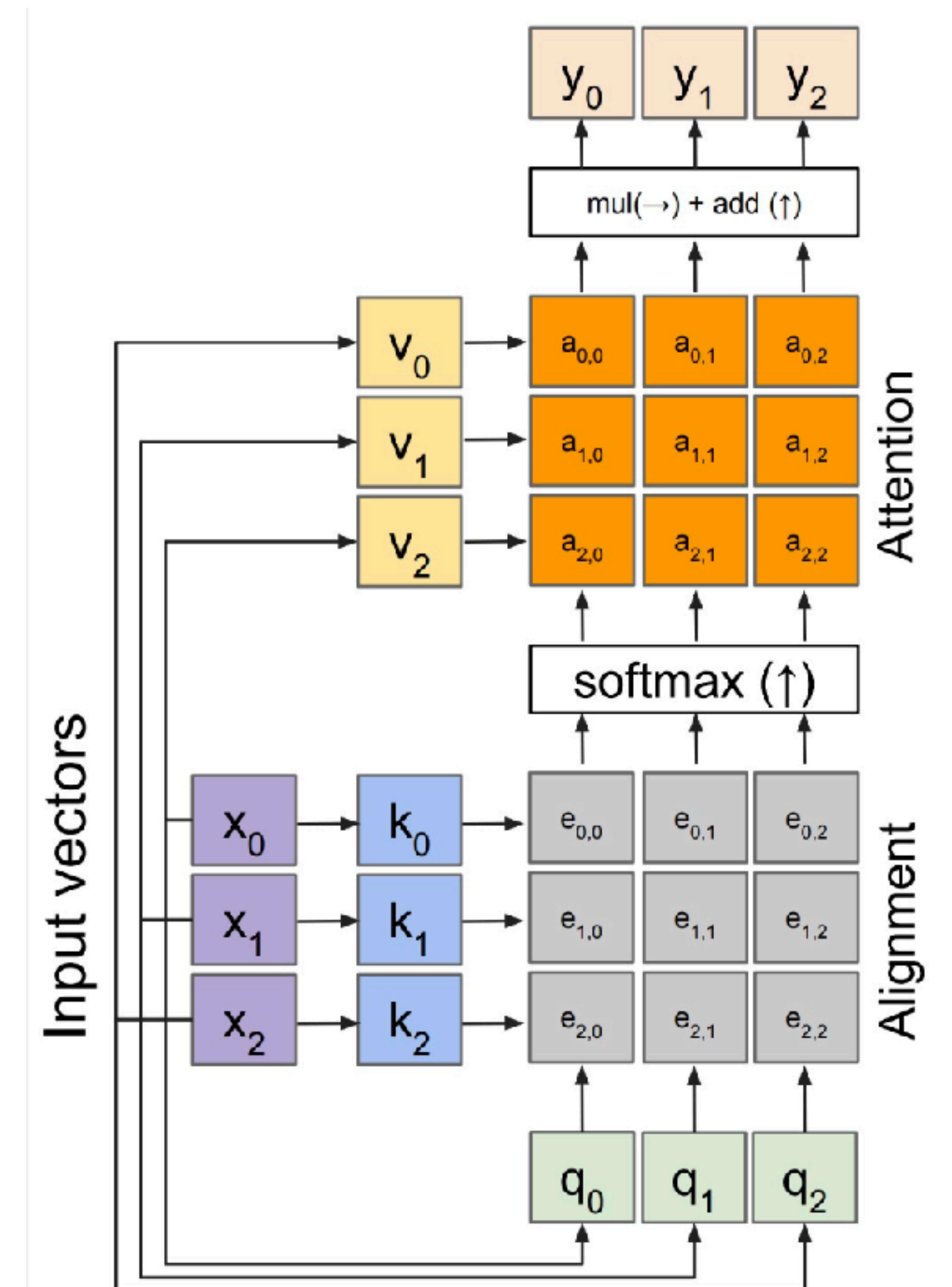
4. **Attention:** Softmax function is applied to the alignment vectors to get attention weights that sum up to 1 (within a column).

$$\mathbf{a} = \text{softmax}(\mathbf{e})$$

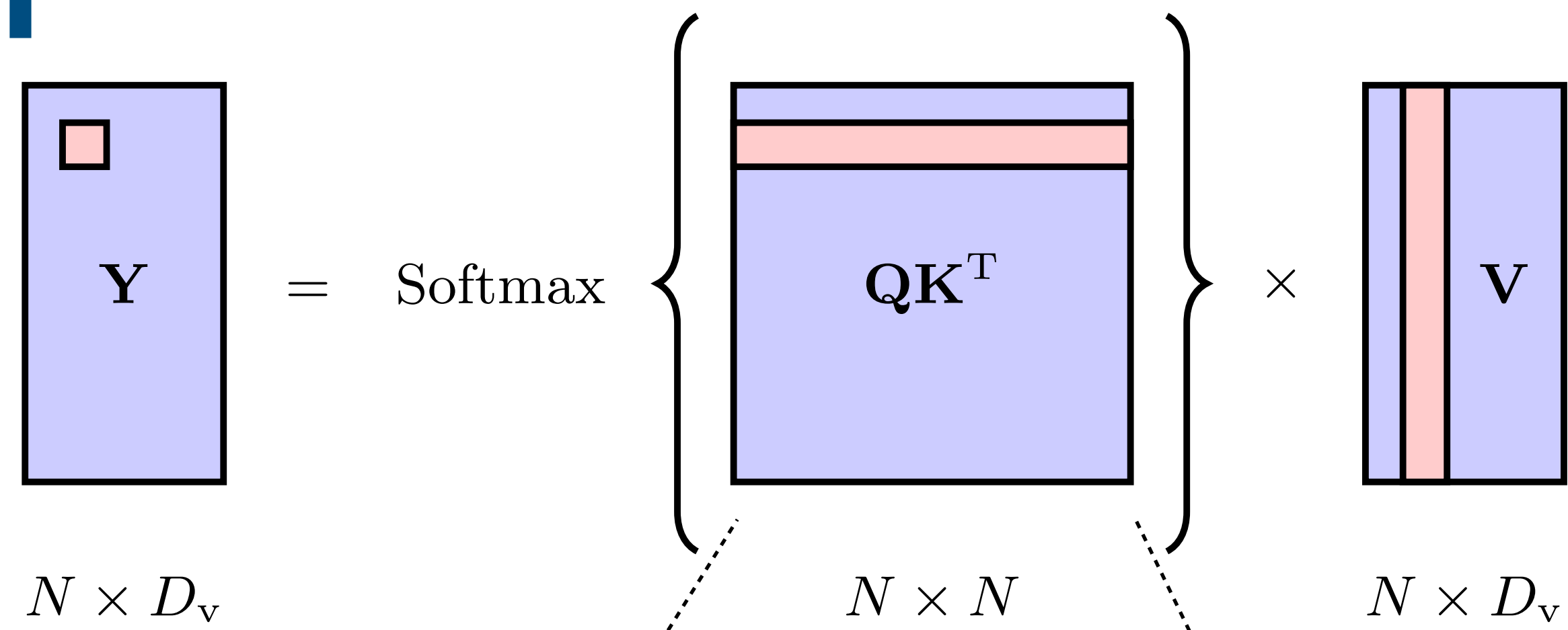
5. **Output:** Attention weights are applied to the value vectors to get the final output.

$$\mathbf{y}_j = \sum_i a_{ij} * \mathbf{v}_i$$

- Putting all the outputs together, it outputs representation of size $N \times D$, a D dimension vector corresponding to each query.

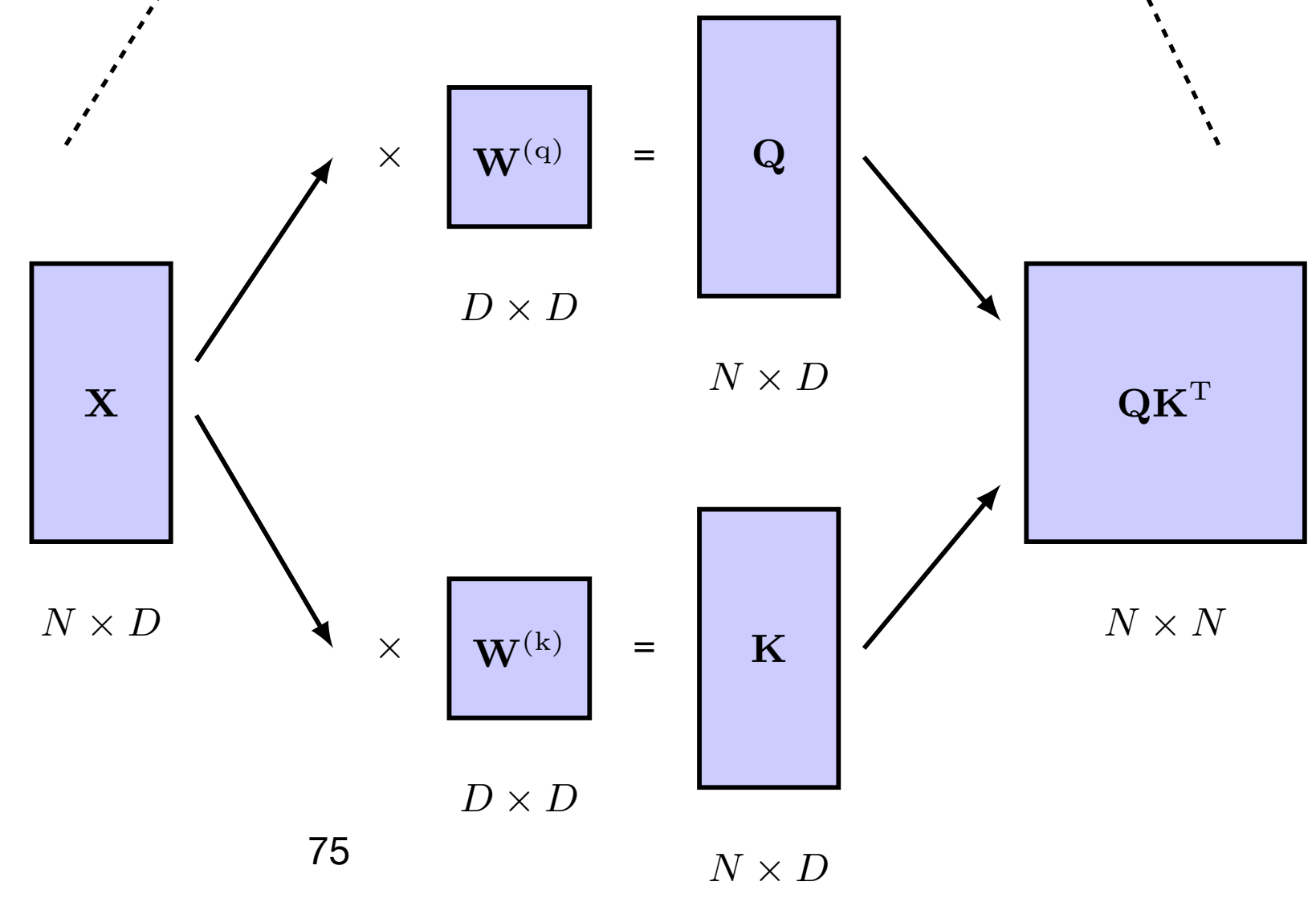


Self Attention



Output
Calculation

$$Y = \text{Softmax}[QK^T]V$$



Questions